International Journal of Forecasting 41 (2025) 321–344

Contents lists available at ScienceDirect

International Journal of Forecasting

journal homepage: www.elsevier.com/locate/ijforecast

Cross-temporal forecast reconciliation at digital platforms with machine learning☆

Jeroen Rombouts [a], Marie Ternes [b], Ines Wilms [b,*]

[a] Essec Business School, Information Systems, Decision Science and Statistics, 3 Av. Bernard Hirsch, 95000 Cergy, France
[b] Maastricht University, School of Business and Economics, P.O. Box 616, 6200 MD Maastricht, The Netherlands

ARTICLE INFO

Article history:
Dataset link: https://github.com/MarieTernes/ml-crosstemporal-reconciliation

Keywords:
Hierarchical time series
Forecast reconciliation
Machine learning
Cross-temporal aggregation
Demand forecasting
Platform econometrics

ABSTRACT

Platform businesses operate on a digital core, and their decision-making requires high-dimensional accurate forecast streams at different levels of cross-sectional (e.g., geographical regions) and temporal aggregation (e.g., minutes to days). It also necessitates coherent forecasts across all hierarchy levels to ensure aligned decision-making across different planning units such as pricing, product, controlling, and strategy. Given that platform data streams feature complex characteristics and interdependencies, we introduce a non-linear hierarchical forecast reconciliation method that produces cross-temporal reconciled forecasts in a direct and automated way through popular machine learning methods. The method is sufficiently fast to allow forecast-based high-frequency decision-making that platforms require. We empirically test our framework on unique, large-scale streaming datasets from a leading on-demand delivery platform in Europe and a bicycle-sharing system in New York City.

© 2024 The Author(s). Published by Elsevier B.V. on behalf of International Institute of Forecasters. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Time series to be forecasted are often naturally part of a hierarchical structure, where higher frequency and granular series are added together to form lower frequency aggregated series. Separate forecasts of each series rarely conserve this hierarchy, and *forecast reconciliation* methods are required. In this paper, we consider novel forecast reconciliation methods for on-demand delivery platforms that require forecasts at different levels of cross-sectional and temporal aggregation. We introduce non-linear forecast reconciliation based on popular machine learning methods to capture the complex interdependencies of platform data.

Platforms such as Uber, Lyft, GrubHub, UberEats, and DoorDash are nowadays omnipresent in the global economy. They operate in high frequency and on a large number of verticals (regions, product categories, etc.). Indeed, their marketplace is typically split up into different geographical regions. Hence, a natural *cross-sectional* aggregation scheme arises from many individual delivery areas over zones towards a few marketplaces. Moreover, a *temporal* aggregation scheme naturally arises since, on the granular end, fast operational decisions (think in terms of minutes) are needed to ensure the platform's service couriers are at the right time and location to serve consumer demand promptly and to determine compensation schemes for couriers through dynamic pricing. On the coarser end, strategic business decisions also require long-term planning since the budget available for each delivery area is set typically using daily demand forecasts. Accurate and coherent, i.e., *reconciled*, demand forecasts across all levels of the cross-sectional and temporal hierarchy are, therefore, key to the business's success and to support aligned decision-making across different planning units.

☆ The numerical results presented in this manuscript were reproduced by the Editor-in-Chief on 21 May 2024.

* Corresponding author.
E-mail address: i.wilms@maastrichtuniversity.nl (I. Wilms).

https://doi.org/10.1016/j.ijforecast.2024.05.008
0169-2070/© 2024 The Author(s). Published by Elsevier B.V. on behalf of International Institute of Forecasters. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Hyndman, Ahmed, Athanasopoulos, and Shang (2011) and Athanasopoulos, Ahmed, and Hyndman (2009) have pioneered the literature to reconcile forecasts; see Hyndman and Athanasopoulos (2021) for a textbook introduction and Hyndman and Athanasopoulos (2014) for an introduction to practitioners given the strong interest from industry. Typical initial applications concern tourism, where demand series are subject to a cross-sectional hierarchy across different geographical regions. However, many other cases appeared more recently; see Caporin, Di Fonzo, and Girolimetto (2023) for a financial application to realized volatility forecasting. While forecast reconciliation methods initially focused on producing coherent forecasts across *cross-sectional* hierarchies, the literature on forecast reconciliation in the *temporal* direction (Athanasopoulos, Hyndman, Kourentzes, & Petropoulos, 2017) as well as combinations of both in *cross-temporal* frameworks is much sparser though in recent years it has proliferated (Athanasopoulos, Hyndman, Kourentzes, & Panagiotelis, 2024b). Early work on cross-temporal forecast reconciliation can be found in Kourentzes and Athanasopoulos (2019), which has then been formalized further in Di Fonzo and Girolimetto (2023a, 2023b) and extended to a probabilistic forecast setting in Girolimetto, Athanasopoulos, Di Fonzo, and Hyndman (2023). We refer the interested reader to Athanasopoulos et al. (2024b) for an extensive review of forecast reconciliation in cross-sectional, temporal, and cross-temporal hierarchies, to Hollyman, Petropoulos, and Tipping (2021) and Di Fonzo and Girolimetto (2024) for the link with the forecast combination literature, and to Athanasopoulos, Hyndman, Kourentzes, and Panagiotelis (2024a) for recent innovations in hierarchical forecasting.

Methodological considerations to forecast reconciliations are discussed in Hyndman, Lee, and Wang (2016), whereas theoretical results are first established by Wickramasuriya, Athanasopoulos, and Hyndman (2019) by putting forward an optimal linear forecast reconciliation method, minimizing the sum of forecast error variances across the collection of time series. Van Erven and Cugliari (2015) offer a game-theoretic approach to forecast reconciliation, Panagiotelis, Athanasopoulos, Gamakumara, and Hyndman (2021) provide a unification of the former two works, and Girolimetto and Di Fonzo (2023b) present a generalization of point and probabilistic forecast reconciliation for general, linearly constrained multiple time series. Traditionally, forecast reconciliation is thus typically achieved by *linearly* combining base forecasts of all the time series in the hierarchy. Recently, however, applications of machine learning algorithms have surfaced to reconcile forecasts in a *non-linear* way.

Machine learning (ML) algorithms nowadays have become an integral component of every forecaster's toolkit, boasting a remarkable track record as demonstrated by various forecasting competitions, such as the M4 and M5 competitions (Makridakis, Spiliotis, & Assimakopoulos, 2020, 2022b), and successful economic applications (e.g., Medeiros, Vasconcelos, Veiga, & Zilberman, 2021). ML methods have become standard for obtaining forecasts as their algorithms demand minimal user intervention– typically limited to tuning parameters –and their sophisticated off-the-shelf implementations are readily available, facilitating seamless operationalization in production environments. Moreover, ML methods offer flexibility by imposing no constraints on the number of features for a given target, and they come with built-in mechanisms for feature selection and for capturing complex dependencies such as non-linearities.

In the forecast reconciliation literature, a cross-sectional ML-based reconciliation scheme is proposed by Spiliotis, Abolghasemi, Hyndman, Petropoulos, and Assimakopoulos (2021), on which our work is mainly built. They first collect base forecasts for all series in the cross-sectional hierarchy and then train a tree-based ML algorithm (random forest or XGBoost) with the bottom-level series as response and one-step-ahead forecasts of all the time series in the hierarchy as input features. The ML-fitted algorithm is hereby used to revise the bottom-level forecasts non-linearly, given that all base forecasts and coherent forecasts across the complete hierarchy are obtained via bottom-up aggregation. In contrast to standard linear reconciliation approaches that reconcile forecasts by minimizing the reconciliation error between the base and reconciled forecasts, the ML-based reconciliation approach is directly geared towards obtaining the minimum forecast combination error to improve out-of-sample forecast accuracy while adhering to coherence. This results in substantial forecast improvements, as demonstrated on two applications to the tourism and retail industry. Additionally, ML-based methods are more flexible than the standard ones as they allow for non-linear combinations of the base forecasts and selectively combine them in a direct and automated way without requiring that the information in the entire hierarchy be used for producing reconciled forecasts.

Anderer and Li (2022) use deep learning and LightGBM tree-based methods to propose a variant of the bottom-up method proposed by Spiliotis et al. (2021), accounting for biases that are difficult to observe at the bottom level. The latter forecasts are adjusted to obtain a higher forecasting accuracy on the upper levels of the hierarchy. Their approach works well on the M5 competition dataset consisting of over ten thousand hierarchical retail sales time series (see Makridakis, Spiliotis, & Assimakopoulos, 2022c and Makridakis, Spiliotis, & Assimakopoulos, 2022a for details). Abolghasemi, Tarr, and Bergmeir (2024) use tree-based methods and lasso regression in their dynamic top-down, bottom-up method applied to fast-moving consumer goods and the M5 competition dataset. Finally, a new stream of research uses neural networks for forecasting and reconciliation; examples of such end-to-end modeling approaches are Theodosiou and Kourentzes (2021) and Wang et al. (2022), which bring promising evidence on various datasets.

Our paper also proposes an ML-based reconciliation approach, extending the work of Spiliotis et al. (2021) in the following ways. From a methodological point of view, we generalize their ML-based forecast reconciliation method for cross-sectional hierarchies to cross-temporal hierarchies. Additionally, considering reconciliation across the temporal direction requires important design choices with respect to the ML model used for

reconciliation since the inputs consist of mixed-frequency time series, namely all temporal frequencies in the temporal hierarchy. Besides, since the ML algorithm is a key backbone in our forecast reconciliation approach, we carefully investigate the effect of (i) the ML algorithm used, thereby considering not only random forest and XGBoost as in Spiliotis et al. (2021) but also LightGBM due to its successful application in the M5 competition; and (ii) tuning hyperparameters to carefully consider the trade-off between improvements in forecast accuracy and computational time. Finally, we consider an extensive set of base forecasts, ranging from academic standards to industry standards (for our platform applications), as well as a combination of several base forecast methods.

From an application viewpoint, we showcase the performance of our forecast reconciliation method on three prime platform demand streaming datasets, thereby bringing novel insights into the performance of forecast reconciliation methods on modern and unique datasets. In our first two platform applications, we use UK demand data (September 2022-September 2023) from a last-mile logistics platform where the bottom level series are 30-min demand series for 117 delivery areas in London (dataset 1) and 24 delivery areas in Manchester (dataset 2), their two largest markets. We aggregate hourly and daily frequency in the temporal dimension and towards zone and/or market level in the cross-sectional dimension. In our third platform application, we use New York demand rental data (January 2023-December 2023) from a bicycle sharing system where the bottom level series are 30-min demand series for six H3 cells– a geographical hexagon unit proposed and used by Uber –in New York City. Here, we consider the set of all possible temporal aggregation orders for a 30-min/1 day temporally grouped time series, and we aggregate towards the New York City level in the cross-sectional dimension.

Platform data have several features distinct from the traditional datasets considered in the forecast reconciliation literature. First, they are high-frequency with, our top, hence lowest frequency, level (i.e., daily) consisting of the bottom level considered in earlier work such as recent M5 competition (other works even consider lower frequency settings such as monthly to yearly data). Indeed, computational speed and fast decision-making at digital platforms are key to their success, requiring intraday forecasts. A second distinctive feature of platform data is their streaming character, which means that they face non-stationarities or shifts, for example, due to expansion, competition among platforms, sales of certain clients, changing customer preferences, or non-regular events such as sports events. The data on Manchester highlight the performance of the ML forecast reconciliation method in the presence of data shifts. We compare our approach with state-of-the-art reconciliation benchmarks from the recent literature.

We obtain several insights based on our applications. First, our main empirical finding is that ML-based forecast reconciliation for platform data can result in substantial forecast accuracy improvements compared to existing linear reconciliation methods. However, ML-based reconciliation is not uniformly superior to linear methods. In fact, for the most important series, i.e., high-frequency area level, random forest-based reconciliation typically performs best. For market-level series, LightGBM-based reconciliation yields the smallest forecast errors. On the other hand, linear reconciliation methods outperform XGBoost-based reconciliation on the bottom-level series. The latter result holds for XGBooost without tuning its hyperparameters to keep computing time low. However, ignoring the huge computational cost, we also study the impact of hyperparameter tuning of the ML algorithms and find, for example, that XGBoost performs at par with random forest in the lowest levels of the hierarchy. A second empirical finding is that ML-based reconciliation seems to work equally well, irrespective of the quality of the base forecasts. Hence, exploring a large model space to obtain base forecasts is unnecessary. Finally, data streams are prone to data shifts, so we compare the forecast reconciliation methods on the Manchester market impacted by a major shift in our evaluation period. The results show that, while the forecast performance of all methods suffers during a data shift, ML-based reconciliation can revert quickly back to pre-shift forecast accuracies.

The rest of the paper is organized as follows. Section 2 introduces the platform data we analyze in this paper. Section 3 presents our ML-based forecast reconciliation approach for cross-temporal hierarchies. Section 4 describes the forecast set-up we use to evaluate the out-of-sample forecast performance of our ML-based reconciliation approach versus state-of-the-art benchmarks, whereas Section 5 presents the results. Finally, Section 6 concludes.

2. Platform data

We describe the three datasets we use to demonstrate the performance of the ML-based reconciliation methods. All are prime examples of platform data; the first two come from an on-demand last-mile logistics platform (Section 2.1), and the third from a bicycle sharing platform (Section 2.2).

2.1. Last-mile logistics platform data

We use two unique datasets of demand for deliveries from a leading on-demand last-mile logistics platform in Europe, Stuart, which connects businesses to a fleet of geographically independent couriers.[1] We analyze UK London and Manchester demand 30-min frequency data from September 5, 2022, to September 24, 2023. London is split into 18 zones to efficiently organize parcel deliveries, constituted by the 117 bottom-level delivery areas. In contrast, Manchester cross-sectionally consists of 24 delivery areas at the bottom level and Manchester as the entire market - given its size, and there is no intermediate

[1] The data are provided to us in the context of ongoing research collaborations. Due to a confidentiality agreement, we cannot distribute or report actual demand data. Demand data across all figures are therefore normalized between 0–100. All analyses were carried out with the original data.

Table 1
Overview of the cross-temporal hierarchy for the three datasets.

Hierarchy	London	Manchester	Citi Bike
Cross-sectional			
number of levels	3	2	2
n_b bottom-level series	117	24	6
n_a upper-level series	19	1	1
n total series	136	25	7
Temporal			
number of levels	3	3	10

level consisting of zones in Manchester. Planning at the delivery area level is of particular importance because it determines the remuneration paid to the couriers and directly helps optimize the platform's efficiency.

Deliveries occur daily between 7am and 11:30pm in all areas.[2] This creates a balanced dataset of 13,090 observations at the 30-min frequency for the 136 (= 1+18+117) time series in London and for each of the 25 (= 1+24) time series in Manchester. For business planning purposes, the series also require temporal aggregation to the hourly and the daily frequency, thereby resulting in temporally aggregated time series that consist of $13,090/2 = 6,545$ and $13,090/34 = 385$ observations respectively. We restrict ourselves to these three temporal frequencies since these are used for decision-making and doing so permits the platform to keep the computational burden under control in a production environment where speed is vital. For the London dataset, the forecast reconciliation consists of three cross-sectional and three temporal layers, implying a 408 distinct time series. The Manchester dataset has two cross-sectional and three temporal layers, implying 75 distinct time series. The cross-temporal hierarchies of both datasets are summarized in Table 1.

London Dataset. Fig. 1 plots demand for the City of London delivery area, London Central zone, and London market at 30-min, hourly, and daily frequencies over the sample period. We observe some typical characteristics of platform data. First, intra-day area data is spiky because of peak demand periods triggered by flash sales, food promotions, and bad weather. These peaks are largely tempered at a daily frequency. Holidays can also cause large drops in delivery demand, particularly noticeable in the end-of-year period. Second, there are strong seasonality patterns at all levels of the hierarchy. Fig. 2 displays these seasonality patterns. The intra-day seasonality (visible from the 30-min and hourly time series) is mainly driven by food deliveries. The bottom row of the plots traces daily seasonality and illustrates that there can be substantial differences between weekends and the rest of the week. In general, London faces higher weekend demand, while the opposite is true for the delivery area of the City of London. This implies heterogeneity in demand patterns at the delivery area level. Unreported plots for other delivery areas reveal that very different dynamics appear.

[2] Occasional overnight demand is integrated in the last 30 min of the business day.

Fig. 3 illustrates this heterogeneity across areas. In the left panel, we visualize the aggregated demand over the sample period for each delivery area. Apart from larger activity above the River Thames, demand volume has no specific spatial pattern; low- and high-volume delivery areas co-exist. The largest volume area is Romford, outside the city center. The City of London and Westminster area volumes, respectively, represent about 15% and 5% of Romford. In the right top panel of Fig. 3, we plot the 30-min average demand throughout the business day for each delivery area. The solid black median line visualizes the lunch and dinner periods well, and the colored area curves stress the large variability, except for early morning demand, which is low overall. The right bottom panel of Fig. 3 zooms into the proportion of zeros across areas. Between 10am and 9pm, most delivery areas have a low proportion of 30-min zero-demand spells. However, Stirling Road (upper green curve) has mostly zero demand throughout the business day. Large diversity appears in the morning and evening hours, with delivery areas that are idle and others that are active.

Manchester Dataset. The streaming platform data for the London market are reasonably stable over time. The Manchester dataset, in contrast, exhibits shifts in the data streams. Fig. 4 displays aggregated demand over the sample period in each delivery area in Manchester. Fig. 5, top panel, illustrates how severe the data shift is on June 14, 2023, in all considered temporal frequencies for the Bolton Central delivery area northwest of Manchester. However, for the entire market of Manchester, the data shift is not apparent, as seen from the bottom panel of Fig. 5.

The Manchester dataset thus offers a compelling scenario to investigate how our ML-based forecast reconciliation procedure performs when facing data shifts. Breaks or data shifts are regularly observed in platform data because new competitors are entering the marketplace, and companies are moving some of their locations to other on-demand platforms.

Finally, since London and Manchester constitute the two largest UK markets of the delivery platform, one might wonder why we keep the two datasets separate as this may hinder the opportunity for further improvement of forecast accuracy. We treat both cities separately since the platform considers the forecasting tasks independently. Indeed, London and Manchester are marketplaces with potential spillover effects between the delivery areas in their market. Each city operates independently by local decision-makers because the business environment, e.g., competitors and demographics, can drastically change from one city to another. The demand forecasts are used to determine salary levels of local independent couriers, which form the market's supply side and are city-specific.

2.2. Bicycle sharing platform data

The third dataset consists of the demand for bicycle rentals across six H3 cells in New York City. The data comes from Citi Bike, owned by the leading American mobility service provider Lyft, and is publicly available at

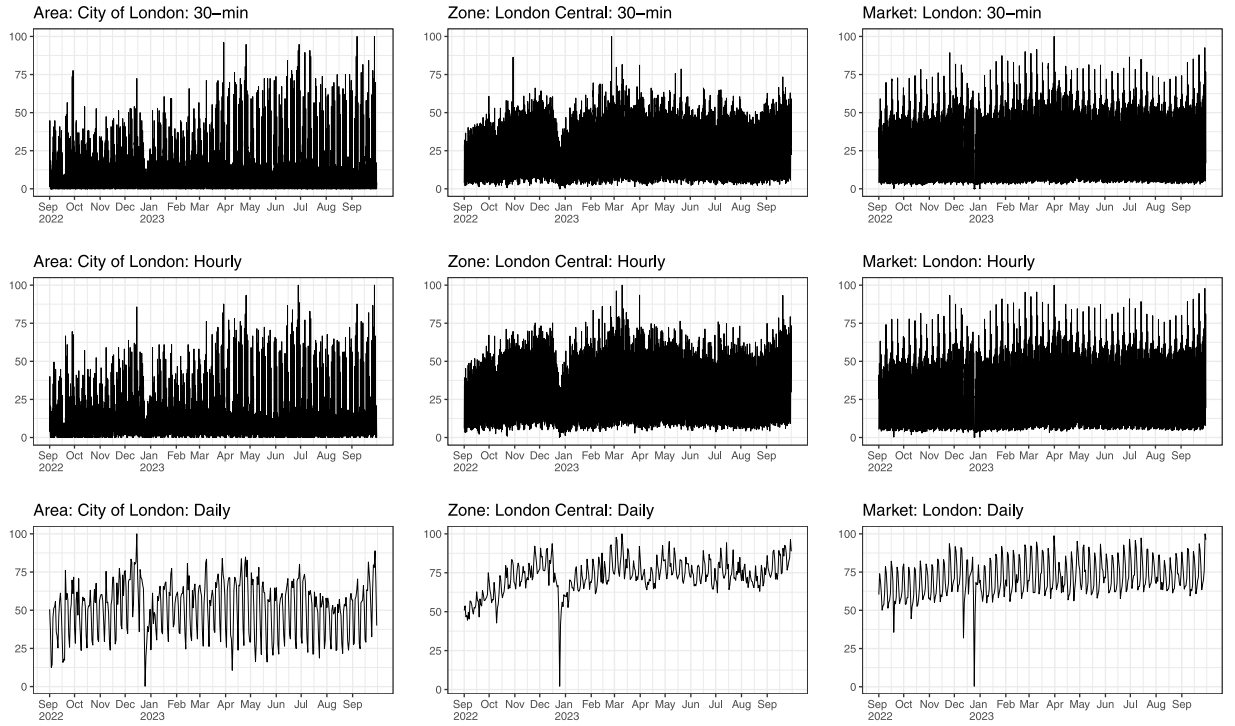


Fig. 1. Time series plots for the full sample of the area City of London belonging to the zone London Central and the market London. Demand data are normalized between 0–100 in each panel. To relate the scale of the series at the different cross-sectional levels (area, zone, market), note for reference that demand at the market level is, on average, 15 times larger than at the zone level, which is, in turn, 18 times higher than at the area level.

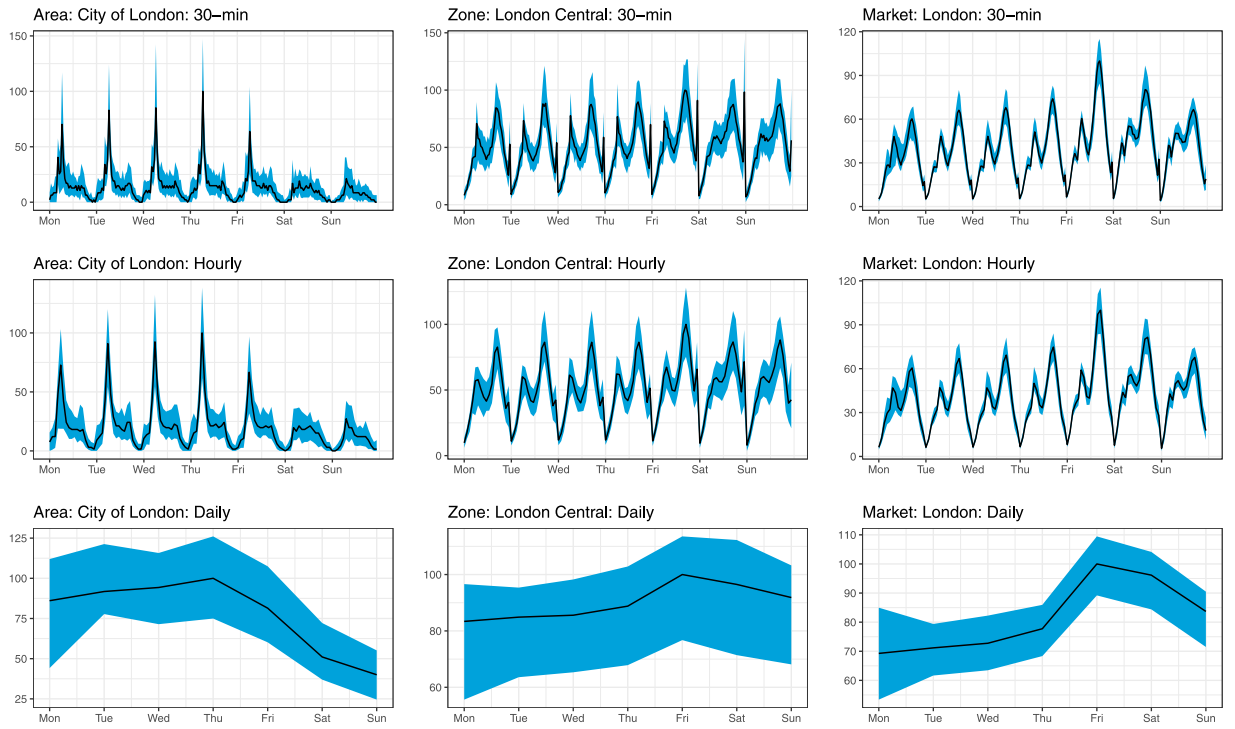


Fig. 2. Time series plots for 30-min, hourly, and daily demand (in the rows) for the area City of London belonging to the zone London Central and the market London (in the columns). The black solid line is the pointwise median. The shaded blue area displays the 5% and 95% pointwise quantiles. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

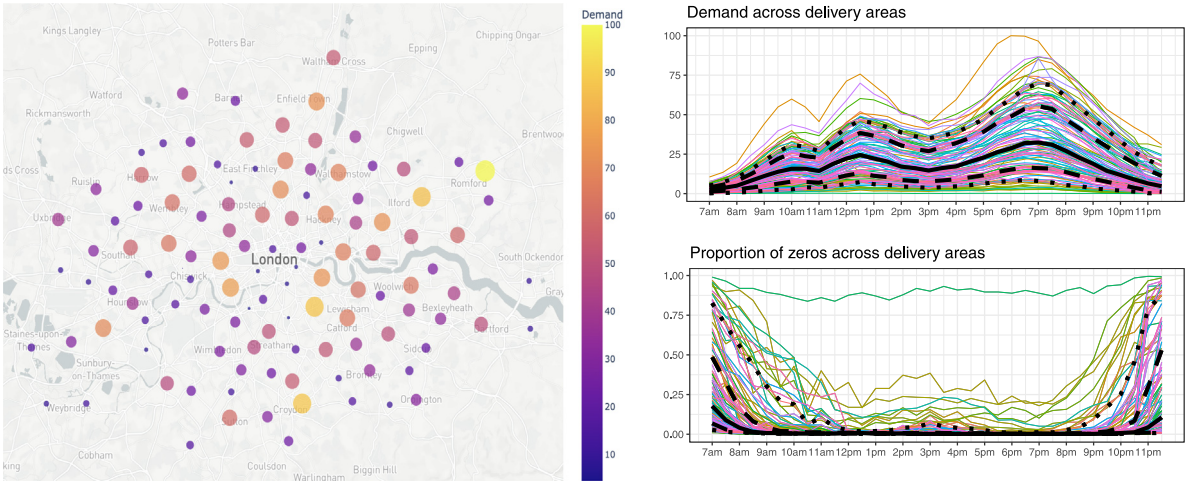


Fig. 3. Left: Aggregated demand in each area (circle) in the London market. Right: Average demand (top) and proportion of zeros (bottom) across areas; the median is displayed in solid black, 10% and 90% quantiles in dotted black, 25% and 75% in dashed black.

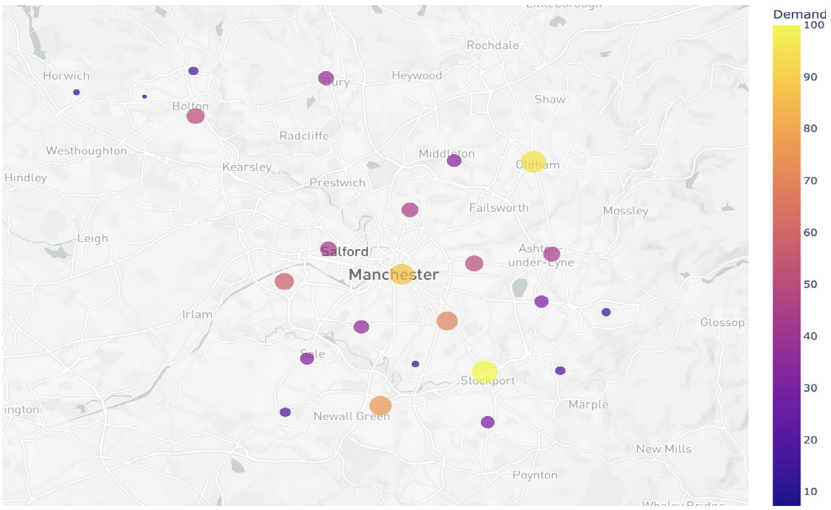


Fig. 4. Aggregated demand in each delivery area in Manchester.

https://citibikenyc.com/system-data. We download data for the year 2023 (2023-citibike-tripdata.zip) and process them to obtain geospatial streaming 30-min time series measuring demand for bicycle rentals across the six H3 cells from 12:00am to 11:30pm. H3 cells, Uber's Hexagonal Hierarchical Spatial Index, are defined using the geospatial indexing system that partitions the world into hexagonal cells.

The cross-temporal hierarchy for our Citi Bike dataset consists of two cross-sectional levels and ten temporal levels, as summarized in Table 1. New York City forms the upper level for the cross-section, whereas the bottom level consists of six H3 cells. In Fig. 6, the left panel displays the aggregate demand over the sample period in each H3 cell, whereas the right panel displays the average 30-min demand in each H3 cell (colored) and in New York City (black) as a whole. For the temporal orders, we consider the set of all possible temporal aggregation orders for a 30-min/1 day temporally grouped time series, namely 30-min, 1-h, 1.5-h, 2-h, 3-h, 4-h, 6-h, 8-h, 12-h and 24-h. In contrast to the hierarchical (nested) temporal structure of the two last-mile delivery data sets, the temporal structure of the Citi Bike data set is grouped since the bottom-level series are aggregated by attributes that are crossed, see Athanasopoulos et al. (2024b) for a discussion.

3. Machine learning based forecast reconciliation

In Section 3.1, we review the cross-temporal framework using the general notation from Athanasopoulos et al. (2024b). In Section 3.2, we introduce our ML-based forecast reconciliation approach for cross-temporal hierarchies.

3.1. Cross-temporal hierarchies

We start with some general notation so that our metho-

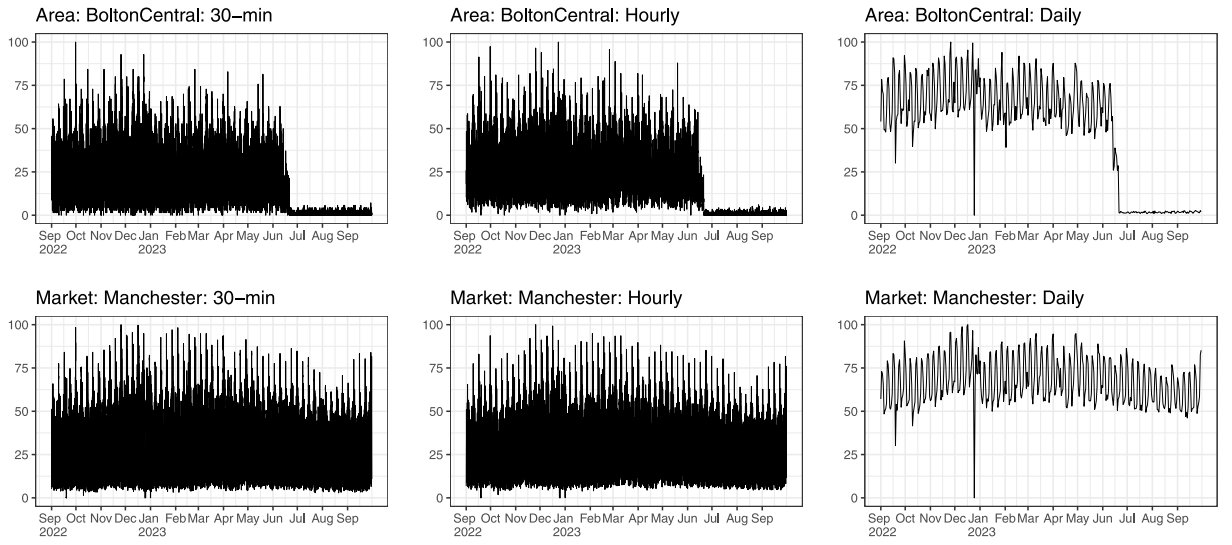


Fig. 5. Time series plots for the full sample of area Bolton Central (top) belonging to the market Manchester (bottom), with a data shift for Bolton Central on June 14, 2023. Demand data are normalized between 0–100 in each panel. To relate the scale of the series at the different cross-sectional levels (area, market), note for reference that demand at the market level is, on average, 15 times larger than at the area level before and 250 times higher after the data shift.

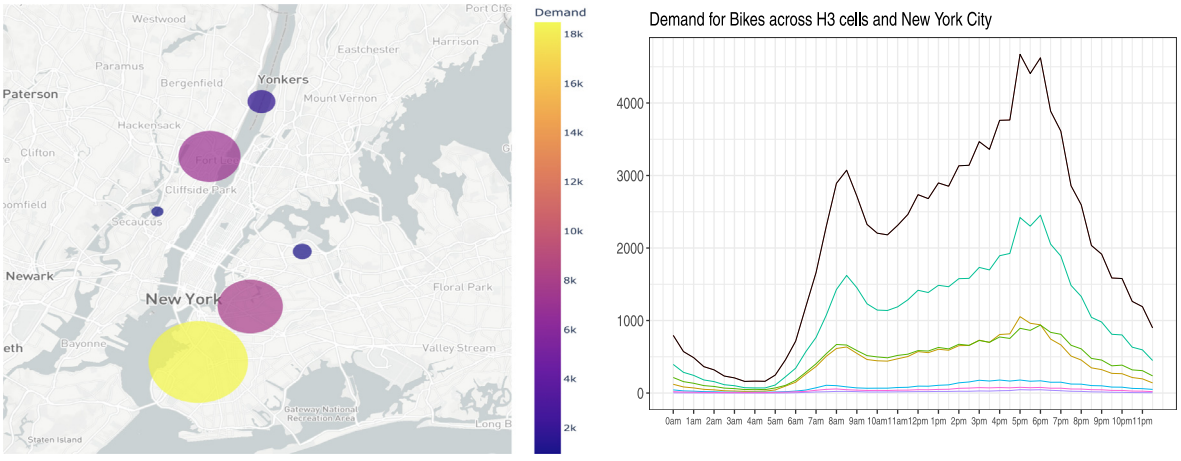


Fig. 6. Left: Aggregated demand (in 1000 s) in each H3 cell (circle). Right: Average demand across the six H3 cells (colored) and New York City (black).

dology for the next section can be widely applied. We denote $y_{b,t}$ for $b = 1, \ldots, n_b$ and $t > 0$ the high-frequency time series observation for bottom level b at time t. The cross-sectional hierarchy is simply obtained by summing over elements in the set of the n_b bottom-level time series. This yields n_a additional high-frequency time series resulting in a total of $n = n_a + n_b$ series.

Regarding the temporal hierarchy, note that in a platform setting, data is streaming, but for simplicity, let us consider $t = 1, \ldots, m$ with m the number of observations to sum over when going from the highest to the lowest sampling frequency in the hierarchy. If we define $k = k_1, \ldots, k_p$ the p temporal aggregation levels, with $k_1 = 1$ and $k_p = m$, then the non-overlapping temporally aggregated series, for each series $i = 1, \ldots, n$ and $k = k_1, \ldots, k_p$, are given by

$$x_{i,j}^{[k]} = \sum_{t=(j-1)k+1}^{jk} y_{i,t} \qquad j = 1, \ldots, m/k.$$

Next, denoting τ as the observation index of the lowest frequency (top temporal level), we can stack these observations in $(m_k \times 1)$ vectors $\boldsymbol{x}_{i,\tau}^{[k]} = \left[x_{i,m_k(\tau-1)+1}^{[k]}, x_{i,m_k(\tau-1)+2}^{[k]}, \ldots, x_{i,m_k\tau}^{[k]}\right]^\top$, where $\top$ denotes the transpose of a vector and $m_k = m/k$.

Finally, the cross-temporal hierarchy is obtained by considering the entire temporal hierarchy for each variable $i = 1, \ldots, n$ of the cross-sectional hierarchy. Notation wise, we collect in a $(\sum_{l=1}^{p} m_{k_l} \times 1)$ vector all temporally aggregated time series $\boldsymbol{x}_{i,\tau} = \left[\boldsymbol{x}_{i,\tau}^{[m]}, \ldots, \boldsymbol{x}_{i,\tau}^{[1]\top}\right]^\top$,

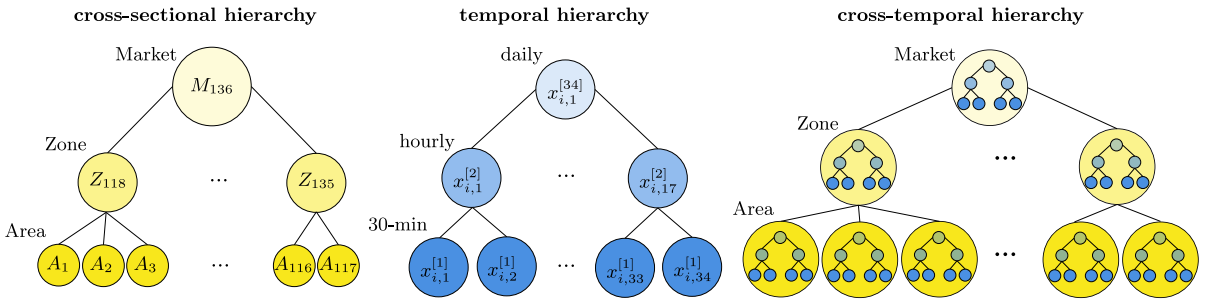


Fig. 7. Cross-sectional (left), temporal (middle), and cross-temporal (right) hierarchy in the platform application.



Fig. 8. Rolling-window procedure for machine-learning based forecast reconciliation.

and then stack all n variables into the vector $\boldsymbol{x}_\tau = \left[\boldsymbol{x}_{1,\tau}^\top, \ldots, \boldsymbol{x}_{n,\tau}^\top\right]^\top$.

In our London platform application, regarding the cross-sectional hierarchy, the bottom level consists of $n_b = 117$ delivery areas that are aggregated into the upper levels consisting of 18 zones (middle) and finally, one market (top level), as visualized in the left panel of Fig. 7. The total number of variables is thus $n = 117 + 18 + 1 = 136$. Regarding the temporal hierarchy, we have three frequencies $p = 3$, (30-min - hourly - daily), as visualized in the middle panel of Fig. 7, $m = 34$ and $k_1 = 1$, $k_2 = 2$ and $k_3 = 34$ so that $\boldsymbol{x}_{i,\tau}^{[1]}$ ($\boldsymbol{x}_{i,\tau}^{[2]}$) collects the $m_1 = 34$ ($m_2 = 17$) 30-min (hourly) demand observations on day τ, and $x_{i,\tau}^{[34]}$ corresponds to the single daily observation for variable i. Finally, the combined cross-temporal hierarchy is displayed in the right panel of Fig. 7. In our Manchester platform application, the temporal hierarchy is the same as that of London. Still, the cross-sectional hierarchy consists of $n_b = 24$ bottom-level series aggregated in a single upper-level series for the market. Finally, for the Citi Bike application, the cross-section consists of $n_b = 6$ bottom-level series that are aggregated in a single upper-level series, whereas for the temporal hierarchy, the set of $p = 10$ temporal aggregation orders is given by $K = \{k_1 = 1, 2, 3, 4, 6, 8, 12, 16, 24, k_p = 48\}$ with $m = 48$.

3.2. Forecast reconciliation via machine learning

We are now ready to introduce our machine learning-based forecast reconciliation approach for cross-temporal hierarchies, extending the work of Spiliotis et al. (2021) for cross-sectional hierarchies.

The general idea behind ML-based reconciliation is to model the relationship between each bottom-level time series $\boldsymbol{x}_{b,\tau}^{[1]}$ ($b = 1, \ldots, n_b$) of the cross-temporal hierarchy and the base forecasts across the entire cross-temporal tree $\widehat{\boldsymbol{x}}_{i,\tau}^{[k]}$ ($i = 1, \ldots, n$ and $k = k_1, \ldots, k_p$). We use a time series rolling-window procedure to construct these base forecasts as visualized in Fig. 8. The rolling-window size of the estimation sample is denoted by Q (length of the blue bars), and the validation sample with forecast horizon by H (length of the green bars); both quantities are expressed in number of observations for the top temporal aggregation level (i.e., the daily time series in our applications). Then, for each cross-sectional series at temporal aggregation level k and rolling window iteration $r = 1, \ldots, R$ (i.e., $R = 4$ in Fig. 8 as used in our last-mile logistics application), we estimate base forecast models on the estimation sample, consisting of $m_k \times Q$ observations, and obtain $m_k \times H$ multistep forecasts, with step-ahead-periods ranging from 1 to $m_k \times H$.

Next, from the observations in the validation sample $\mathcal{V}$ consisting of observations $\mathcal{V} = \{Q + 1, \ldots, Q + R \times H\}$, we construct the response vector and features matrix for the ML-model which can be generally written as

$$\boldsymbol{y}_{b,\mathcal{V}} = f_b(\widehat{\boldsymbol{X}}_{b,\mathcal{V}}) + \boldsymbol{\varepsilon}_{b,\mathcal{V}}, \tag{1}$$

where $f_b(\cdot)$ denotes the forecast function to be trained for bottom-level series b, see Section 4.3 for specifications, and $\boldsymbol{\varepsilon}_{b,\mathcal{V}}$ represents the error term. Note that every bottom-level series in the cross-temporal hierarchy (i.e., 30-min series in our applications) is predicted by a separate ML model, thereby allowing the forecast reconciliation approach to adapt to different patterns in each series. The response variable $\boldsymbol{y}_{b,\mathcal{V}}$ in (1) is simply given by the $m_1 \times R \times H$ observations of the bottom-level series for variable b; hence

$$\boldsymbol{y}_{b,\mathcal{V}} = \left[\boldsymbol{x}_{b,Q+1}^{[1]\top}, \ldots, \boldsymbol{x}_{b,Q+RH}^{[1]\top}\right]^\top.$$

As features matrix to the ML-model, the most natural generalization of Spiliotis et al. (2021) to the cross-temporal framework would be to use the same features matrix for each bottom-level series b, namely $\widehat{\boldsymbol{X}}_{b,\mathcal{V}} = \widehat{\boldsymbol{X}}_{\mathcal{V}} =$ (see Box I), which contains all the base forecasts for all the series in the cross-temporal hierarchy on the validation sample, where $\otimes$ denotes the Kronecker product and $\mathbf{1}$

$$\begin{bmatrix} \hat{\boldsymbol{x}}^{[1]}_{1,Q+1} & \cdots & \hat{\boldsymbol{x}}^{[1]}_{n,Q+1} & \hat{\boldsymbol{x}}^{[k_2]}_{1,Q+1} \otimes \mathbf{1}_{k_2} & \cdots & \hat{\boldsymbol{x}}^{[k_2]}_{n,Q+1} \otimes \mathbf{1}_{k_2} & \cdots & \hat{x}^{[k_p]}_{1,Q+1} \otimes \mathbf{1}_{k_p} & \cdots & \hat{x}^{[k_p]}_{n,Q+1} \otimes \mathbf{1}_{k_p} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ \hat{\boldsymbol{x}}^{[1]}_{1,Q+RH} & \cdots & \hat{\boldsymbol{x}}^{[1]}_{n,Q+RH} & \hat{\boldsymbol{x}}^{[k_2]}_{1,Q+RH} \otimes \mathbf{1}_{k_2} & \cdots & \hat{\boldsymbol{x}}^{[k_2]}_{n,Q+RH} \otimes \mathbf{1}_{k_2} & \cdots & \hat{x}^{[k_p]}_{1,Q+RH} \otimes \mathbf{1}_{k_p} & \cdots & \hat{x}^{[k_p]}_{n,Q+RH} \otimes \mathbf{1}_{k_p} \end{bmatrix}$$

Box I.

$$\begin{bmatrix} \hat{\boldsymbol{x}}^{[1]}_{1,Q+1} & \cdots & \hat{\boldsymbol{x}}^{[1]}_{b-1,Q+1} & \hat{\boldsymbol{x}}^{[1]}_{b+1,Q+1} & \cdots & \hat{\boldsymbol{x}}^{[1]}_{n,Q+1} & \hat{\boldsymbol{x}}^{[1]}_{b,Q+1} & \hat{\boldsymbol{x}}^{[k_2]}_{b,Q+1} \otimes \mathbf{1}_{k_2} & \cdots & \hat{\boldsymbol{x}}^{[k_p]}_{b,Q+1} \otimes \mathbf{1}_{k_p} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \hat{\boldsymbol{x}}^{[1]}_{1,Q+RH} & \cdots & \hat{\boldsymbol{x}}^{[1]}_{b-1,Q+RH} & \hat{\boldsymbol{x}}^{[1]}_{b+1,Q+RH} & \cdots & \hat{\boldsymbol{x}}^{[1]}_{n,Q+RH} & \hat{\boldsymbol{x}}^{[1]}_{b,Q+RH} & \hat{\boldsymbol{x}}^{[k_2]}_{b,Q+RH} \otimes \mathbf{1}_{k_2} & \cdots & \hat{\boldsymbol{x}}^{[k_p]}_{b,Q+RH} \otimes \mathbf{1}_{k_p} \end{bmatrix}$$

Box II.

is a column vector of ones. Due to the frequency mismatch, each of the lower-frequency series at level k_a for $a = 2, \ldots, p$ are repeated respectively k_a times. Note, however, that $\widehat{\boldsymbol{X}}_{\mathcal{V}}$ contains $p \times n$ features in its columns, which may quickly become large, and this curse of dimensionality may lead to a serious computational burden in practice. To solve this issue, we suggest a more compact bottom-series specific features matrix as an alternative, namely $\widehat{\boldsymbol{X}}_{b,\mathcal{V}} =$ (see Box II), which contains a selection of base forecasts on the validation sample. More specifically, the first set of predictors, highlighted in yellow, exploits information in the cross-sectional hierarchy and contains the n_b highest frequency (i.e., 30-min in our applications) base forecasts for all bottom-level variables, as well as the n_a highest-frequency base forecasts for the upper-level series in the cross-temporal hierarchy (i.e., zone and market for London; market for Manchester and Citi Bike). The second set of predictors highlighted in blue exploits information in the temporal hierarchy and contains all temporal base forecasts for the bottom-level variable b for which we are constructing our ML model. The number of features in the columns of the matrix $\widehat{\boldsymbol{X}}_{b,\mathcal{V}}$ is now substantially reduced to $n + p - 1$ features, but does rest on the assumption that the temporal gain in reconciling forecasts comes from the focal variable's temporally aggregated series to keep this second set of predictors compact. Finally, note the central role of the bottom-level variable b itself (highlighted in yellow and blue), whose base forecast we aim to improve upon in this reconciliation step. In the remainder of the paper, we use this more compact features matrix to keep the computational burden feasible when displaying the main results.[3]

[3] We ran some small-scale experiments for the last-mile delivery applications and did not find a substantial gain in out-of-sample forecast performance from using the complete features matrix $\widehat{\boldsymbol{X}}_{\mathcal{V}}$ instead of the more compact features matrix $\widehat{\boldsymbol{X}}_{b,\mathcal{V}}$, thereby providing empirical support for choosing the latter in these applications. For the Citi Bike dataset, we find an improvement in out-of-sample forecast performance from using $\widehat{\boldsymbol{X}}_{\mathcal{V}}$ instead of $\widehat{\boldsymbol{X}}_{b,\mathcal{V}}$ for the H3 cells but not for the market, see Section 5.2.2 for further details.

Finally, to reconcile the actual out-of-sample demand forecasts of horizon $h = 1, \ldots, H$ for forecast evaluation purposes, we use the complete sample up to $N = Q + R \times H$ to obtain the $m_k \times H$ multistep forecasts (with step-ahead-periods ranging from 1 to $m_k \times H$) for each cross-sectional series at temporal aggregation level k. This is visualized in the bottom part of Fig. 8, where the gray bar represents the training sample (of length N) on which the base forecast models are estimated, and the red bar (of length H) represents the test set on which we evaluate out-of-sample forecast performance. The out-of-sample base forecasts $\widehat{\boldsymbol{x}}^{[1]}_{i,N+h}$ ($i = 1, \ldots, n$ and $h = 1, \ldots, H$), and $\widehat{\boldsymbol{x}}^{[k]}_{b,N+h}$ ($b = 1, \ldots, n_b$, $h = 1, \ldots, H$, and $k = k_2, \ldots, k_p$) are used as inputs to the previously estimated ML models (one for each bottom-level series) to provide the revised bottom-level forecasts $\widetilde{\boldsymbol{x}}^{[1]}_{b,N+h}$ ($b = 1, \ldots, n_b$ and $h = 1, \ldots, H$) of the cross-temporal hierarchy. As a last step, these revised bottom-level forecasts are aggregated as explained in Section 3.1 to obtain coherent forecasts across the complete hierarchy.

4. Design of the forecast study

We start by discussing in Section 4.1 the forecast set-up we use to compare the forecast reconciliation methods. Section 4.2 discusses the metrics we use to evaluate out-of-sample forecast performance. Section 4.3 summarizes the base forecast models all forecast reconciliation methods rely on, together with the machine learning methods used by our procedure to obtain the reconciled forecasts. Finally, Section 4.4 outlines the benchmark forecast methods against which we compare the performance of our procedure.

4.1. Forecast set-up

We use a standard rolling-window approach to compare the forecast performance of our reconciliation method against its benchmarks. We first describe the set-up used for London and Manchester, then summarize the differences in the set-up for Citi Bike at the end.

London and Manchester. We use the period February 20, 2023 to September 24, 2023 (31 weeks) as the test sample. In each rolling window iteration, we use the most recent 24 weeks of data to estimate the base models on the estimation set (first 20 weeks), and the most recent four weeks of data to estimate the ML model on the validation set. Hence, in line with the notation used in Fig. 8, we take $Q = 140$ days, $H = 7$ days and $R = 4$. By taking $H = 7$, it should be noted that we combine base forecasts with different forecast uncertainty, as the latter typically increases with the forecast horizon. Finally, since the demand data in our platform applications consists of non-negative integers, we ensure this by rounding the base forecasts before using them as inputs in the ML model, i.e., $\hat{x}_{i,t}^{[k]\star} = \max(0, \lfloor \hat{x}_{i,t}^{[k]} \rceil)$, we similarly round the bottom-level reconciled forecasts before aggregating them.

In the next step, we combine the estimation and validation sample ($N = 168$ days) to estimate the base models and obtain final out-of-sample base forecasts and reconciled forecasts on the test set. As forecast horizon, we consider one week of data (i.e., we obtain $m_k \times H$ multistep forecasts where the step-ahead periods range from 1 to $m_k \times H$ for temporal aggregation level k) as this corresponds to the planning horizon for the compensation schemes of couriers through dynamic pricing which crucially relies on demand forecasts as inputs. Hence, for the 30-min series, we obtain $34 \times 7 = 238$ forecasts, for the hourly series, we obtain $17 \times 7 = 119$ forecasts, and for the daily series, we obtain $H = 7$ forecasts. This concludes one iteration in the outer rolling window used for forecast evaluation. We move one week ahead in the next iteration and repeat the same steps. We roll forward until we reach our sample's end, resulting in 31 outer rolling windows.

Figure S.1 in the online supplement provides a visualization of iteration r and $r+1$ in the outer rolling window. We highlight the estimation (blue bars) and validation (green bars) sets for the inner rolling window used to construct the inputs to the ML model (see Section 3.2). Due to our inner rolling-window set-up, the training set (blue bar) and forecast horizon (green bar) of the last three (inner) rolling windows in iteration r overlap with the first three (inner) rolling windows in iteration $r + 1$, as indicated with the shading. This offers a clear computational advantage: beyond iteration $r = 1$, we can re-use the base forecasts from the previous iteration for the first three weeks. We need to fit the forecasting model only once to obtain the base forecasts for the fourth week to construct the validation set.

Citi Bike. We use June 18, 2023, to December 31, 2023, as a test sample and take $Q = 140$ days, $H = 1$ day, $R = 28$ days, and $N = 168$ days. As forecast horizon, we thus consider the natural choice $H = 1$ day, which is of great interest in many other fields where forecast reconciliation has been applied. In each iteration of the outer-rolling window, we then roll one day forward, resulting in 197 outer-rolling windows.

4.2. Evaluating forecast performance

To evaluate the forecast performance of the various forecast methods on the three datasets, we use the Weighted Absolute Percentage Error (WAPE), alternatively referred to as the Weighted Average Percentage Error, and known as the normalized version of the mean absolute error (MAE), aka as MAD/Mean ratio as firstly considered by Hoover (2006), where MAD is a synonym for MAE. The WAPE accuracy index for each temporal factor k and cross-sectional time series $i = 1, \ldots, n$ is given by

$$\text{WAPE}_i^{[k]} = \frac{\sum_{j=1}^{T_{\text{test}}/k} |A_{i,j}^{[k]} - F_{i,j}^{[k]}|}{\sum_{j=1}^{T_{\text{test}}/k} A_{i,j}^{[k]}},$$

which measures the overall deviation of the forecasted values $F_{i,j}^{[k]}$ from the actual values $A_{i,j}^{[k]}$, for all time points j in the test set of size T_{test}/k where T_{test} denotes the number of high-frequency (i.e., 30-min) observations in the test set. Note that the WAPE can be multiplied by 100 to obtain a percentage value. We then average these WAPEs across all cross-sectional units belonging to the same cross-temporal level; for instance, the overall WAPE for the bottom-level series (i.e., 30-min for the n_b areas) is then given by $\text{WAPE}^{[1]} = (1/n_b) \sum_{b=1}^{n_b} \text{WAPE}_b^{[1]}$.

The WAPE is a popular forecast metric to evaluate performance at delivery platforms: (i) its scale independence facilitates comparisons across heterogeneous areas as well as different time/geographical aggregation levels; (ii) WAPE is agnostic to zeros as it is very unlikely that the denominator is zero, thereby making it a suitable metric even when the demand in an area is low or intermittent; and (iii) WAPE emphasizes accuracy for items with larger demand, thereby prioritizing accurate forecasting of demand during busy periods.

Note that, as a robustness check, we also report results for the Mean Absolute Scaled Error (MASE), defined as

$$\text{MASE}_i^{[k]} = \frac{\frac{1}{m_k H} \sum_{j=m_k N+1}^{m_k(N+H)} |A_{i,j}^{[k]} - F_{i,j}^{[k]}|}{\frac{1}{m_k(N-7)} \sum_{j=7m_k+1}^{m_k N} |A_{i,j}^{[k]} - A_{i,j-7m_k}^{[k]}|},$$

for each temporal factor k and cross-sectional time series $i = 1, \ldots, n$. Here, the absolute (out-of-sample) errors are scaled based on the in-sample MAE from a "naive" forecast method in which the forecast for each future time slot of next week is the actual value for the same specific time slot and day of this week. While the WAPE assumes that the mean is stable over time, making it sensitive to trend, seasonality, or other patterns in the data, the MASE is unaffected by such patterns, for example, the seasonal pattern that repeats itself every week for a specific hour of the day and day of the week in the datasets we consider. We refer the interested reader to Hyndman (2006), Hyndman and Koehler (2006) for a more elaborate discussion.

4.3. Base forecasts and machine learning methods

Our procedure allows practitioners to choose their preferred base forecast models and ML methods to perform the ML-based reconciliation. This paper considers four base forecast models and three popular ML methods.

Base Forecasts. We consider one popular industry forecast model for platform data, two all-round time series models for forecasting univariate time series, and

a forecast combination of the former three as ensemble, base forecast model. The first base forecast model is a simple yet often-used model in the platform industry, which we will label as "Naive" since the forecast for a specific slot (for instance, 30-min, hourly, or daily) of next week is simply the value observed in the same slot and day of the previous week. The following two base forecast models are often used in the hierarchical forecasting literature to obtain base forecasts: the class of SARIMA and exponential smoothing models. Full details on the base forecast methods are available in the online supplement.

Machine Learning Methods. We consider three popular ML models: random forest, XGBoost, and LightGBM. The former two are also considered in Spiliotis et al. (2021) whereas the latter has recently shown great success on the M5 competition (Makridakis et al., 2022a) consisting of an application with a hierarchical retail sales time series, making all suitable candidates for our platform data. More details on the ML models are available in the online supplement.

We report the main results for all three ML methods based on their standard implementations with default tuning parameters (see the online supplement). We do this, first to avoid excessive computing times on our streaming platform data, but second, and importantly, also because off-the-shelf implementations of tree-based methods often attain excellent performance across a variety of settings (e.g., Januschowski et al., 2022). In Section 5.1.3, we investigate the improvements in forecast performance one can obtain by tuning these ML models.

Finally, when reporting our main results, we use the sum of squared errors as a loss function to train all ML models. However, our framework allows practitioners to use another loss function suited to the data. We also investigated the performance of the ML-based reconciliation methods when using the Tweedie loss function instead of the standard squared loss since the former showed superior performance in the M5 competition (Januschowski et al., 2022; Makridakis et al., 2022a) on forecasting hierarchical time series as it is specifically geared towards sparse (zero-inflated) target data, which some of the considered delivery areas also display (see Fig. 3). However, using a Tweedie loss instead of the regular squared loss function did not improve the forecast results of our proposed ML reconciliation methods on the considered data and is hence omitted but available from the authors upon request.

4.4. Benchmark reconciliation methods

We compare our new ML forecast reconciliation procedure against the base forecasts and five state-of-the-art linear reconciliation benchmarks for cross-temporal hierarchies, as summarized in Table 2 and implemented in the `FoReco` package (Girolimetto & Di Fonzo, 2023a) in `R` (R Core Team, 2023). Apart from the Bottom-Up method, the reconciliation step requires specifying how to estimate the covariance matrix of the in-sample fit errors since the benchmark methods are geared towards minimizing reconciliation errors instead of our procedure, which is geared towards minimizing forecast combination errors. For the tcs, cst, and ite benchmark methods, the temporal reconciliation covariance matrix estimator is the series variance scaling matrix (thf_comb = "wlsv") which is the diagonal matrix that contains the estimated variances of the in-sample residuals across each level. The cross-sectional reconciliation covariance matrix estimator is a shrinkage covariance matrix of the in-sample residuals (hts_comb = "shr"), where the off-diagonal elements are shrunk towards zero. The oct method's cross-temporal based covariance matrix estimator simply contains in-sample residual variance on the main diagonal (csts_comb = "wlsv"). Full details on the cross-temporal benchmark methods are available in Di Fonzo and Girolimetto (2023a).[4] Finally, note that possible negative reconciled values for the linear reconciliation benchmarks are dealt with similarly to the ML-based reconciliation methods.

5. Results

This section is divided into two parts. We present the results for the last-mile delivery platform and Citi Bike.

5.1. Results for last-mile logistics platform applications

In Section 5.1.1, we first discuss the overall forecast performance of the forecast reconciliation methods for London and Manchester. We then zoom into the London dataset and present detailed insights on our ML-based reconciliation approach (Section 5.1.2), and discuss the trade-off between computing time and forecast accuracy when incorporating hyperparameter tuning into our ML-based reconciliation proposal (Section 5.1.3).

5.1.1. Overall forecast performance

We start by discussing the forecast performance on the London dataset and then highlight how shifts in the demand data streams affect the performance of the reconciliation methods on the Manchester dataset.

London. Table 3 summarizes the forecast accuracy, in terms of WAPE, across the complete cross-temporal hierarchy for the different combinations of base forecasts and reconciliation methods.[5] Overall, all methods find

[4] There exist two different implementations of the iterative cross-temporal forecast reconciliation "ite". The first reconciliation step in each iteration can either start from the temporal or the cross-sectional reconciliation. We estimate both procedures, but only report on the latter since it is the most competitive on the considered datasets.

[5] Note that for the Naive base method, the base and bottom-up forecasts are identical, and no results are reported for the linear forecast reconciliation methods since the original base forecasts are already reconciled. Indeed, the standard linear reconciliation methods reconcile forecasts by minimizing the reconciliation error between the base and reconciled forecasts, making this step redundant. In contrast, the non-linear ML-based reconciliation methods are geared towards minimizing forecast combination errors to improve out-of-sample forecast performance. Then, there is value in starting from the naive base forecasts since they might not be "inherently reconciled" in the "optimal way" (in terms of out-of-sample forecast performance), yet are attractive to use in a production environment where computational speed is key since they do not require estimation time, unlike SARIMA and ETS.

Table 2
Overview of cross-temporal forecast reconciliation benchmarks.

Name	Description
Bottom Up	Simple aggregation of bottom level forecasts for any cross-temporal hierarchy
tcs	First-temporal-then-cross-sectional cross-temporal forecast reconciliation - (Kourentzes & Athanasopoulos, 2019)
cst	First-cross-sectional-then-temporal cross-temporal forecast reconciliation - (Di Fonzo & Girolimetto, 2023a)
ite	Iterative cross-temporal forecast reconciliation starting with cross-sectional reconciliation - (Di Fonzo & Girolimetto, 2023a)
oct	Optimal combination cross-temporal forecast reconciliation in a single reconciliation step - (Di Fonzo & Girolimetto, 2023a)

Notes: This table summarizes the benchmark cross-temporal forecast reconciliation methods.

Table 3
London forecast reconciliation results.

Base Forecasts	Forecast Method	Delivery Area			Zone			Market		
		30min	hour	day	30min	hour	day	30min	hour	day
Naive	Base	0.3576	0.2703	0.1017	0.1475	0.1196	0.0638	0.0793	0.0756	0.0559
	Random forest	0.2762	0.2162	0.0976	0.1271	0.1081	0.0636	0.0819	0.0782	0.0576
	XGBoost	0.3017	0.2338	0.0993	0.1352	0.1132	0.0654	0.0841	0.0796	0.0588
	LightGBM	0.2898	0.2252	0.0991	0.1321	0.1110	0.0650	0.0842	0.0796	0.0587
ETS	Base	0.3180	0.2654	0.0837	0.1935	0.1769	0.0539	0.1621	0.1521	0.0471
	Bottom Up	0.3180	0.2682	0.1755	0.1842	0.1716	0.1378	0.1523	0.1501	0.1313
	tcs	0.3105	0.2591	0.1587	0.1769	0.1641	0.1290	0.1440	0.1420	0.1229
	cst	0.3070	0.2557	0.1536	0.1699	0.1570	0.1191	0.1349	0.1331	0.1117
	ite	0.3112	0.2606	0.1617	0.1782	0.1659	0.1316	0.1454	0.1439	0.1251
	oct	0.3099	0.2577	0.1545	0.1765	0.1633	0.1267	0.1435	0.1411	0.1206
	Random forest	0.2809	0.2229	0.1064	0.1360	0.1192	0.0729	0.0959	0.0933	0.0673
	XGBoost	0.3161	0.2545	0.1189	0.1439	0.1249	0.0735	0.0920	0.0889	0.0634
	LightGBM	0.2960	0.2374	0.1164	0.1387	0.1209	0.0737	0.0926	0.0896	0.0645
SARIMA	Base	0.3118	0.2584	0.0894	0.1793	0.1656	0.0568	0.1490	0.1445	0.0526
	Bottom Up	0.3118	0.2605	0.1637	0.1808	0.1682	0.1339	0.1511	0.1489	0.1286
	tcs	0.3049	0.2524	0.1486	0.1742	0.1612	0.1253	0.1436	0.1416	0.1201
	cst	0.3013	0.2488	0.1424	0.1679	0.1549	0.1164	0.1351	0.1334	0.1100
	ite	0.3051	0.2533	0.1508	0.1750	0.1627	0.1276	0.1446	0.1431	0.1221
	oct	0.3043	0.2510	0.1443	0.1718	0.1583	0.1210	0.1408	0.1385	0.1164
	Random forest	0.2801	0.2206	0.1002	0.1305	0.1132	0.0657	0.0883	0.0861	0.0586
	XGBoost	0.3195	0.2528	0.1109	0.1385	0.1182	0.0659	0.0845	0.0815	0.0551
	LightGBM	0.2933	0.2327	0.1067	0.1320	0.1134	0.0641	0.0832	0.0806	0.0538
Forecast Combination	Base	0.2950	0.2365	0.0850	0.1502	0.1333	0.0538	0.1134	0.1089	0.0485
	Bottom Up	0.2950	0.2358	0.1289	0.1493	0.1333	0.0991	0.1120	0.1097	0.0938
	tcs	0.2925	0.2349	0.1215	0.1470	0.1310	0.0962	0.1093	0.1073	0.0913
	cst	0.2909	0.2331	0.1182	0.1438	0.1277	0.0908	0.1046	0.1027	0.0853
	ite	0.2931	0.2359	0.1238	0.1483	0.1328	0.0984	0.1111	0.1095	0.0933
	oct	0.2917	0.2336	0.1182	0.1454	0.1290	0.0928	0.1070	0.1046	0.0879
	Random forest	0.2758	0.2161	0.1018	0.1285	0.1108	0.0717	0.0871	0.0844	0.0666
	XGBoost	0.3026	0.2355	0.1011	0.1336	0.1129	0.0671	0.0836	0.0803	0.0601
	LightGBM	0.2900	0.2260	0.0999	0.1301	0.1103	0.0664	0.0831	0.0799	0.0593

Notes: This table shows forecast accuracy measured in weighted absolute percentage error (WAPE). The best forecast reconciliation results for each base forecast (Naive, ETS, SARIMA, Forecast Combination) are highlighted in gray. Table 2 defines the linear benchmark methods.

forecasting easier higher up in the hierarchy, i.e., lower temporal frequency and larger geographic areas, which confirms the general findings in the literature. For the set of considered base forecasts and reconciliation methods, WAPE falls from around 30% to 5% for 30-min delivery areas and the daily London hierarchy levels.

The ML-based forecast reconciliation results using random forest consistently outperform all benchmarks at the delivery area and zone levels at all frequencies. For example, using 30-min frequency SARIMA base forecasts, we obtain a WAPE of 0.3013 for the cst method (i.e., the best linear forecast reconciliation method) compared to 0.2801 for random forest-based reconciliation (i.e., the best non-linear-forecast reconciliation method). However, ML-based forecast reconciliation does not systematically dominate linear methods, such as XGBoost, with a WAPE

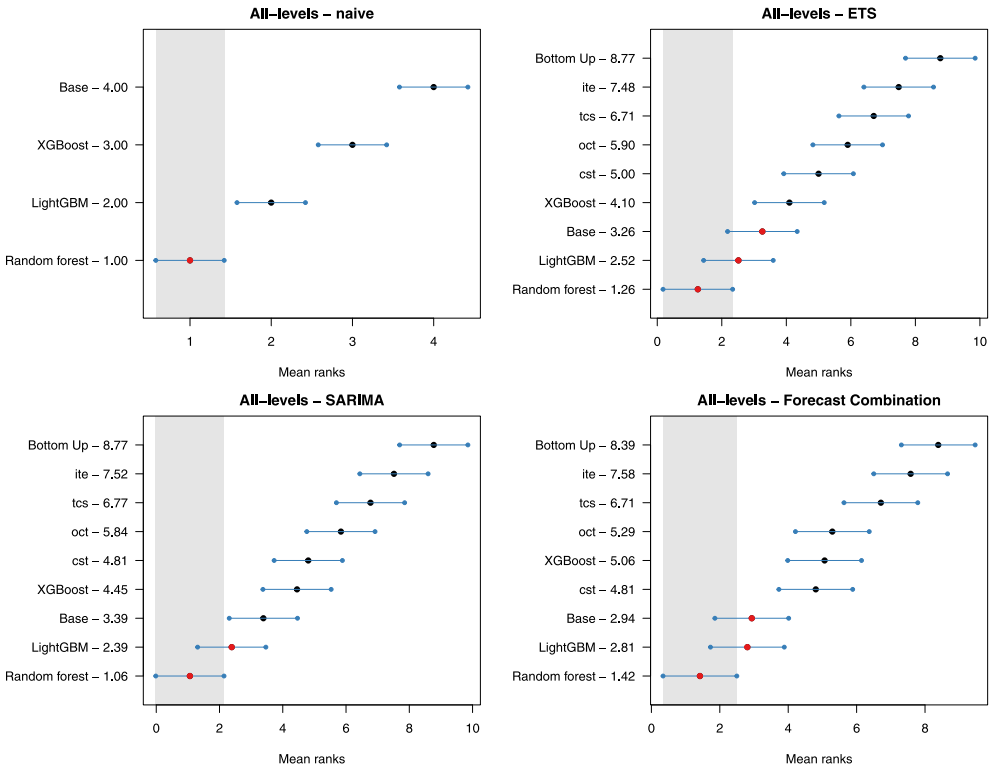


Fig. 9. London: MCB test with a confidence level of 0.95, using all levels in the hierarchy. Methods that do not overlap with the gray zone, indicating the confidence interval of the best method, are significantly worse than the best.

of 0.3195. Furthermore, ML-based forecast reconciliation seems insensitive to the choice of base forecasts. The Naive, ETS, and SARIMA results are qualitatively the same. For example, in the case of random forest, the respective WAPEs are very close with 0.2762, 0.2809, and 0.2801, respectively.

To assess the significance of our findings, we report in Fig. 9 the results of the accuracy ranking based on multiple comparisons with the best (MCB) test, a methodology popularized in the forecasting literature since Koning, Franses, Hibon, and Stekler (2005). We compute the MCB test across all levels in the cross-temporal hierarchy and report results for each of the four base forecast methods. To this end, we use the R-package `tsutils` (Kourentzes, Svetunkov, & Schaer, 2023) with function `nemenyi()` to compute the MCB test. The test ranks the performance of the examined methods across the series, which is forecasted by comparing their average ranks by considering a critical difference determined through a confidence interval. For each base forecast panel in Fig. 9, the methods that do not overlap with the best-ranked procedure (as highlighted by the gray zone) are significantly worse than the best. Visual inspection of the results confirms that random forest reconciliation dominates the other procedures.

The important business question is whether ML-based forecast reconciliation helps improve the base forecast methods for high-frequency (i.e., 30-min and hourly) delivery area levels because these forecasts are direct inputs for automated decision-making. The answer is yes, and the forecast gains are between 10 and 15%. As an illustration for random forest reconciliation with ETS base forecasts, the WAPE is 0.2809 compared to 0.3180 for non-reconciled base forecasts. It also turns out that these high-frequency delivery area level gains become even stronger at the zone and market levels. The forecast gains mount to even 40% at the market level. For example, the London 30-min WAPEs decrease from 0.1621 to 0.0959 with ETS base forecasts and random forest reconciliation. Here, the linear reconciliation methods only deliver marginal accuracy gains, with the best (cst) WAPE being 0.1349. However, we also note that the smallest WAPEs are obtained at the London level using Naive base forecasts for 30-min and hourly frequencies.

Finally, in the bottom panel of Table 3, we provide forecast combination results by combining the base forecasts with equal weights. We find this strategy slightly pays off compared to the best-performing base methods. For example, at the 30-min delivery area level, the random forest ML methods for the base forecasts are 0.2762, 0.2809, and 0.2801, respectively, while the forecast combination yields 0.2758. The gains are stronger overall for the linear reconciliation methods than the non-linear ones, yet the ML-based reconciliation methods outperform the former. Still, in practice, we generally do not know which base forecast method will perform best out-of-sample. Moreover, this likely varies across the different levels of the cross-temporal hierarchy. Then, a forecast combination base forecast approach forms an

Table 4
Manchester forecast reconciliation results.

Period	Forecast Method	Delivery Area			Market		
		30min	hour	day	30min	hour	day
Before shift	Base	0.3549	0.2922	0.1100	0.1884	0.1832	0.0815
	Bottom Up	0.3549	0.2945	0.1879	0.1949	0.1905	0.1630
	tcs	0.3459	0.2837	0.1734	0.1847	0.1794	0.1533
	cst	0.3435	0.2807	0.1680	0.1794	0.1735	0.1460
	ite	0.3464	0.2840	0.1743	0.1855	0.1800	0.1549
	oct	0.3470	0.2846	0.1731	0.1848	0.1797	0.1520
	Random forest	0.3226	0.2523	0.1146	0.1216	0.1141	0.0764
	XGBoost	0.3650	0.2885	0.1315	0.1205	0.1121	0.0743
	LightGBM	0.3418	0.2706	0.1300	0.1196	0.1118	0.0759
During shift	Base	0.8124	0.7687	0.4634	0.2091	0.2034	0.0615
	Bottom Up	0.8124	0.7587	0.5923	0.2092	0.2059	0.1805
	tcs	0.7991	0.7683	0.5926	0.1965	0.1926	0.1642
	cst	0.7986	0.7651	0.5747	0.1885	0.1841	0.1520
	ite	0.8107	0.7710	0.5971	0.1997	0.1956	0.1678
	oct	0.7885	0.7606	0.5789	0.1910	0.1875	0.1581
	Random forest	0.8004	0.7346	0.5885	0.1110	0.1022	0.0646
	XGBoost	0.8330	0.7534	0.5683	0.1128	0.1017	0.0633
	LightGBM	0.7516	0.6818	0.5230	0.1101	0.1002	0.0658
After shift	Base	0.7166	0.6402	0.1610	0.1776	0.1740	0.0524
	Bottom Up	0.7166	0.6464	0.4506	0.1825	0.1786	0.1526
	tcs	0.6969	0.6446	0.3694	0.1682	0.1631	0.1364
	cst	0.7023	0.6484	0.3381	0.1615	0.1558	0.1256
	ite	0.7049	0.6521	0.3553	0.1699	0.1646	0.1381
	oct	0.6925	0.6402	0.3755	0.1650	0.1601	0.1322
	Random forest	0.4206	0.3300	0.1457	0.0992	0.0904	0.0465
	XGBoost	0.4863	0.3874	0.1464	0.1007	0.0896	0.0485
	LightGBM	0.4386	0.3488	0.1551	0.0980	0.0880	0.0474

Notes: This table shows forecast accuracy measured in weighted absolute percentage error (WAPE) for SARIMA base forecasts. The best forecast reconciliation results for the periods Before shift (before June 14, 2023), After shift (after July 14, 2023), and During shift (period in between) are highlighted in gray. Table 2 defines the linear benchmark methods.

effective practice to obtain accurate and robust base forecasts, which can still be further improved via ML-based reconciliation to obtain coherent forecasts.

Manchester. Table 4 summarizes the forecast accuracy for the cross-temporal hierarchy for SARIMA base forecasts and all reconciliation methods; the other base forecast results are similar and available on request. We report results for the period before the data shift (i.e., before June 14), after the data shift (i.e., after July 14), and the one month in between characterizing the data shift. There are several interesting findings for the delivery area level of the hierarchy. First, for the "Before shift" period, we see that random forest reconciliation for delivery areas outperforms other methods with the same relative magnitudes found in the London market where no data shifts occur. For example, random forest and cst WAPEs are 0.3226 and 0.3435, respectively. Second, we see that the volatile "During shift" period WAPEs at the delivery area level all more than double compared to the stable period before. The LightGBM ML reconciliation method performs best with a WAPE of 0.7516. Third, the "After shift" period is generally characterized by better accuracies than during the shift, but the sizes of the gains are much stronger for the ML-based reconciliation methods. To illustrate this magnitude, consider cst with WAPEs of 0.7986 and 0.7023 and random forest WAPEs of 0.8004 and 0.4206, respectively, for the "During" and "After shift" periods. Fourth, the un-reconciled "After shift" SARIMA base forecasts are of low quality, with WAPEs still double the size compared to the "Before shift" period. This can be expected given that parameter estimates are based on historical data subject to the shift. However, our ML-based reconciliation method can transform these low-quality projections into forecasts yielding WAPEs much closer to the period before the shift. At the market level part of the hierarchy, we observe only small differences in forecasting accuracy over the three periods. This is expected since the aggregation of delivery areas, not all subject to a data shift, results in a smoother market-level time series, see Fig. 5, bottom panel. The LightGBM method consistently has the smallest WAPEs at the 30-min and hourly frequency levels.

To sum up, random forest forecast reconciliation performs particularly well, but not all ML methods perform better than linear methods. The strongest gains are obtained for the 30-min and hourly frequencies. Forecast combination, i.e., averaging the base forecasts, is (marginally) beneficial. These main findings are, overall, qualitatively the same when we measure forecast accuracy in terms of the MASE, as can be seen from Tables A.1 and A.2 in Appendix A. Random forest remains the top-performing reconciliation method for the bottom levels of the cross-temporal hierarchy across the London and Manchester datasets. LightGBM becomes even more favorable to use — compared to random forest — for

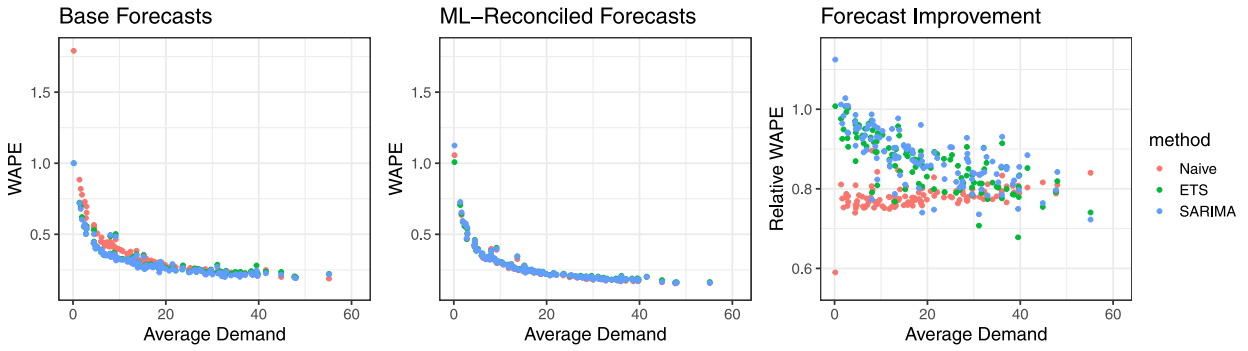


Fig. 10. Forecast performance measured in WAPE for base forecasts (left), for ML-reconciled forecasts via random forest (middle) and relative improvement in WAPE of the latter over the former versus average 30-min demand for the different delivery areas (dots) in London.

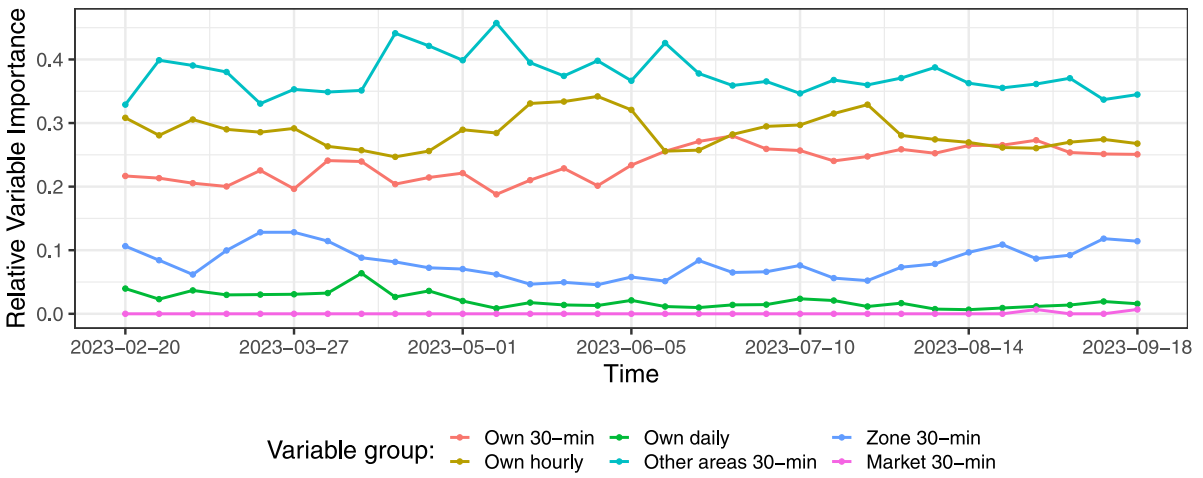


Fig. 11. Variable importance in random forest ML model with Naive base forecasts for the delivery area City of London.

the upper levels of the cross-temporal hierarchy when using the MASE accuracy index instead of WAPE. For Manchester, in the "After shift" period, the margin by which the ML-based reconciliation methods outperform the linear ones is less outspoken in terms of MASE than WAPE.

5.1.2. ML-based reconciliation insights

The previous results show that random forest-based forecast reconciliation outperforms the considered benchmarks, especially at the more granular levels in the hierarchy. To gain more insights, we investigate the link between forecast accuracy and the size of the delivery areas on the London dataset. In addition, we investigate what drives forecast performance by computing SHAP values, considered the current state-of-the-art method for interpreting ML models; see Molnar (2022) for an introduction.

Fig. 10 plots the forecast performance (WAPE, vertical axis) of each delivery area (dot) based on its average demand size (horizontal axis). The left panel shows this relationship using un-reconciled base forecasts, and we find a smirk-shaped pattern with many WAPEs above 0.5. The Naive base forecasts are, overall, worse than ETS and SARIMA when the average demand is below 20. The middle figure shows the random forest forecast reconciliation results, and the smirk flattens out, with hardly any noticeable difference between the base forecast methods and a few areas having WAPEs above 0.5. The right figure highlights that random forest-based forecast reconciliation can "correct" bad quality base forecasts as the relative WAPEs of the ML-reconciled forecasts over the base forecasts are mostly below 1.

Next, for each random forest model with Naive base forecasts in the (outer) rolling-window set-up, we compute SHAP values (Lundberg & Lee, 2017; Shapley et al., 1953) for all observations in the test set. To this end, we use the package `treeshap` (Komisarczyk et al., 2024) in R, a fast implementation for tree ensemble models (Lundberg, Erion, & Lee, 2018). Fig. 11 shows the resulting relative variable importance plots by variable group for the City of London as an illustration. Given the large number of features used in the ML model, we focus on the 36 highest-ranked features in terms of variable importance (which together account for more than 80% of the overall variable importance). Overall, the proportions are stable over the test sample. While the City of London's own 30-min and hourly streams count for a sizable portion in

Table 5
London forecast reconciliation results with hyperparameter tuning.

Forecast Method		Time (s)	Delivery Area			Zone			Market		
			30 min	h	day	30 min	h	day	30 min	h	day
Random forest	default	7	0.2801	0.2206	0.1002	0.1305	0.1132	0.0657	0.0883	0.0861	0.0586
Random forest	tuned	400	0.2784	0.2198	0.1033	0.1324	0.1155	0.0692	0.0911	0.0889	0.0621
XGBoost	default	1	0.3195	0.2528	0.1109	0.1385	0.1182	0.0659	0.0845	0.0815	0.0551
XGBoost	tuned	600	0.2790	0.2209	0.1032	0.1314	0.1144	0.0652	0.0886	0.0864	0.0568
LightGBM	default	1	0.2933	0.2327	0.1067	0.1320	0.1134	0.0641	0.0832	0.0806	0.0538
LightGBM	tuned	900	0.2801	0.2221	0.1034	0.1334	0.1165	0.0668	0.0917	0.0895	0.0589

Notes: This table shows forecast accuracy measured in weighted absolute percentage error (WAPE) for SARIMA base forecasts. Computing times are approximate, measured in seconds, and recorded on a Windows 10 Enterprise LTSC machine with an Intel Core i5 2.4 GHz processor.

Table 6
Citi Bike forecast reconciliation results for the H3 cells.

Base Forecasts	Forecast Method	Temporal Frequency									
		30min	1h	1.5h	2h	3h	4h	6h	8h	12h	24h
Naive	Base	0.2883	0.2675	0.2583	0.2525	0.2434	0.2365	0.2286	0.2199	0.2132	0.1981
	Random forest	0.2682	0.2530	0.2456	0.2400	0.2317	0.2241	0.2136	0.2014	0.1962	0.1775
	XGBoost	0.2951	0.2775	0.2682	0.2620	0.2513	0.2426	0.2305	0.2157	0.2099	0.1879
	LightGBM	0.2882	0.2723	0.2639	0.2583	0.2484	0.2408	0.2278	0.2154	0.2089	0.1869
ETS	Base	0.3699	0.3382	0.3314	0.3246	0.3089	0.2677	0.2482	0.2383	0.2237	0.1552
	Bottom Up	0.3699	0.3593	0.3542	0.3498	0.3408	0.3305	0.3183	0.3052	0.2958	0.2555
	tcs	0.3453	0.3338	0.3283	0.3237	0.3136	0.3016	0.2902	0.2734	0.2620	0.2180
	cst	0.3549	0.3445	0.3391	0.3348	0.3262	0.3149	0.3042	0.2894	0.2795	0.2375
	ite	0.3552	0.3445	0.3391	0.3347	0.3258	0.3146	0.3037	0.2892	0.2795	0.2367
	oct	0.3481	0.3366	0.3311	0.3264	0.3162	0.3042	0.2926	0.2765	0.2653	0.2210
	Random forest	0.2606	0.2446	0.2367	0.2306	0.2213	0.2115	0.1989	0.1908	0.1827	0.1654
	XGBoost	0.2822	0.2642	0.2541	0.2475	0.2369	0.2271	0.2125	0.2034	0.1941	0.1738
	LightGBM	0.2742	0.2580	0.2489	0.2430	0.2328	0.2240	0.2096	0.1994	0.1924	0.1718
SARIMA	Base	0.3467	0.3110	0.3022	0.2982	0.2706	0.2321	0.2110	0.2004	0.1700	0.1546
	Bottom Up	0.3467	0.3353	0.3295	0.3248	0.3137	0.3019	0.2868	0.2728	0.2583	0.2169
	tcs	0.3176	0.3045	0.2977	0.2920	0.2784	0.2636	0.2487	0.2334	0.2171	0.1756
	cst	0.3197	0.3068	0.3000	0.2939	0.2804	0.2652	0.2496	0.2353	0.2185	0.1766
	ite	0.3182	0.3051	0.2981	0.2920	0.2782	0.2628	0.2470	0.2323	0.2158	0.1738
	oct	0.3174	0.3043	0.2976	0.2920	0.2785	0.2637	0.2488	0.2335	0.2173	0.1760
	Random forest	0.2605	0.2445	0.2361	0.2303	0.2201	0.2086	0.1947	0.1866	0.1773	0.1589
	XGBoost	0.2801	0.2624	0.2518	0.2453	0.2332	0.2232	0.2077	0.1984	0.1903	0.1680
	LightGBM	0.2715	0.2550	0.2459	0.2398	0.2287	0.2188	0.2037	0.1948	0.1861	0.1648
Forecast Combination	Base	0.2942	0.2660	0.2578	0.2532	0.2364	0.2109	0.2017	0.1911	0.1758	0.1527
	Bottom Up	0.2942	0.2820	0.2759	0.2719	0.2641	0.2544	0.2443	0.2336	0.2253	0.1951
	tcs	0.2799	0.2667	0.2602	0.2557	0.2470	0.2363	0.2264	0.2150	0.2051	0.1743
	cst	0.2853	0.2728	0.2665	0.2624	0.2540	0.2437	0.2338	0.2230	0.2132	0.1830
	ite	0.2869	0.2743	0.2680	0.2638	0.2554	0.2453	0.2352	0.2245	0.2150	0.1848
	oct	0.2793	0.2661	0.2596	0.2549	0.2461	0.2352	0.2253	0.2138	0.2037	0.1726
	Random forest	0.2521	0.2366	0.2288	0.2237	0.2156	0.2076	0.1964	0.1894	0.1817	0.1647
	XGBoost	0.2756	0.2580	0.2482	0.2419	0.2316	0.2240	0.2113	0.2033	0.1953	0.1763
	LightGBM	0.2692	0.2535	0.2443	0.2389	0.2295	0.2222	0.2094	0.2023	0.1948	0.1752

Notes: This table shows forecast accuracy measured in weighted absolute percentage error (WAPE). The best forecast reconciliation results for each base forecast (Naive, ETS, SARIMA, Forecast Combination) are highlighted in gray. Table 2 defines the linear benchmark methods.

the 20%–30% range, the other areas 30-min streams count together for almost 40%. Zone and market level, or own daily information, contributes only marginally.

5.1.3. Computational aspects and hyperparameter tuning

Our main results above are obtained by training the ML algorithms with their default hyperparameters as provided by the software packages. Doing so saves substantial computing time, an important consideration when deploying forecast reconciliation in a production environment. However, in this section, we investigate whether the performance of the ML-based forecast reconciliation methods can be further improved by hyperparameter tuning. To this end, we use the common practice of grid search to tune the hyperparameters of the random forest algorithm since there are only a few hyperparameters. In contrast, we use Bayesian optimization to tune the XGBoost and LightGBM algorithms' hyperparameters, as the latter have a large amount of hyperparameters. Details on the tuning process are provided in the online supplement.

Table 5 reports results for the ML-based reconciliation methods– default and tuned –with SARIMA base forecasts

Table 7
Citi Bike forecast reconciliation results for the market.

Base Forecasts	Forecast Method	Temporal Frequency									
		30min	1h	1.5h	2h	3h	4h	6h	8h	12h	24h
Naive	Base	0.2270	0.2243	0.2226	0.2213	0.2179	0.2150	0.2097	0.2043	0.1999	0.1891
	Randomforest	0.2225	0.2190	0.2166	0.2143	0.2102	0.2057	0.1976	0.1875	0.1832	0.1695
	XGBoost	0.2338	0.2295	0.2263	0.2237	0.2183	0.2133	0.2052	0.1943	0.1885	0.1732
	LightGBM	0.2356	0.2315	0.2287	0.2266	0.2215	0.2176	0.2080	0.1971	0.1914	0.1751
ETS	Base	0.3839	0.3044	0.2978	0.2928	0.2751	0.2386	0.2166	0.2115	0.1954	0.1469
	Bottom Up	0.3474	0.3446	0.3430	0.3409	0.3328	0.3260	0.3150	0.3031	0.2993	0.2737
	tcs	0.3116	0.3084	0.3063	0.3039	0.2950	0.2862	0.2760	0.2578	0.2507	0.2241
	cst	0.3144	0.3114	0.3091	0.3066	0.2980	0.2890	0.2791	0.2611	0.2553	0.2265
	ite	0.3218	0.3188	0.3166	0.3142	0.3057	0.2972	0.2869	0.2708	0.2659	0.2373
	oct	0.3176	0.3145	0.3125	0.3100	0.3013	0.2929	0.2826	0.2654	0.2600	0.2319
	Random forest	0.2154	0.2104	0.2075	0.2044	0.1979	0.1912	0.1804	0.1724	0.1689	0.1562
	XGBoost	0.2231	0.2180	0.2146	0.2115	0.2048	0.1989	0.1876	0.1790	0.1759	0.1609
	LightGBM	0.2201	0.2153	0.2120	0.2097	0.2026	0.1975	0.1856	0.1779	0.1750	0.1622
SARIMA	Base	0.2915	0.2796	0.2775	0.2779	0.2451	0.2100	0.1988	0.1909	0.1596	0.1573
	Bottom Up	0.2870	0.2830	0.2802	0.2774	0.2661	0.2541	0.2423	0.2224	0.2097	0.1817
	tcs	0.2747	0.2705	0.2673	0.2640	0.2520	0.2392	0.2262	0.2090	0.1939	0.1648
	cst	0.2716	0.2672	0.2639	0.2601	0.2475	0.2342	0.2206	0.2058	0.1889	0.1591
	ite	0.2748	0.2704	0.2670	0.2634	0.2509	0.2378	0.2239	0.2086	0.1920	0.1627
	oct	0.2749	0.2707	0.2675	0.2642	0.2523	0.2395	0.2266	0.2093	0.1944	0.1651
	Random forest	0.2143	0.2087	0.2054	0.2023	0.1947	0.1870	0.1751	0.1683	0.1619	0.1490
	XGBoost	0.2155	0.2099	0.2058	0.2027	0.1959	0.1901	0.1782	0.1706	0.1664	0.1510
	LightGBM	0.2133	0.2080	0.2041	0.2007	0.1938	0.1876	0.1764	0.1693	0.1643	0.1499
Forecast Combination	Base	0.2604	0.2339	0.2304	0.2291	0.2113	0.1912	0.1844	0.1768	0.1613	0.1498
	Bottom Up	0.2522	0.2497	0.2474	0.2460	0.2400	0.2333	0.2239	0.2128	0.2076	0.1889
	tcs	0.2402	0.2374	0.2351	0.2335	0.2272	0.2190	0.2107	0.1985	0.1911	0.1712
	cst	0.2434	0.2407	0.2384	0.2368	0.2306	0.2225	0.2140	0.2024	0.1950	0.1742
	ite	0.2489	0.2464	0.2442	0.2427	0.2366	0.2287	0.2198	0.2089	0.2021	0.1814
	oct	0.2393	0.2364	0.2342	0.2324	0.2261	0.2178	0.2098	0.1972	0.1895	0.1695
	Randomforest	0.2065	0.2023	0.1998	0.1979	0.1931	0.1874	0.1785	0.1742	0.1677	0.1580
	XGBoost	0.2135	0.2090	0.2063	0.2040	0.1991	0.1930	0.1842	0.1791	0.1729	0.1622
	LightGBM	0.2124	0.2082	0.2051	0.2039	0.1983	0.1935	0.1835	0.1790	0.1740	0.1620

Notes: This table shows forecast accuracy measured in weighted absolute percentage error (WAPE). The best forecast reconciliation results for each base forecast (Naive, ETS, SARIMA, Forecast Combination) are highlighted in gray. Table 2 defines the linear benchmark methods.

on the London dataset. The random forest algorithm with default tuning parameters performs best in most of our previous results, and its tuned version also performs well. This is a great advantage when implementing the forecasting procedure in a high-dimensional streaming data environment with frequent updates. In contrast to random forest, the XGBoost algorithm greatly benefits from tuning at the high-frequency delivery area level. Here, the loss decreases from 0.3196 to 0.2790, a reduction of about 13%, and XGBoost is now at the same accuracy level as random forest. The differences between default and tuning are marginal for the zone and market hierarchies. Finally, the LightGBM algorithm is only mildly impacted, with small tuning improvements at the delivery area level.

While hyperparameter tuning does result in a marginal gain in forecast accuracy, it is important to investigate the computational burden that comes with it. Table 5 reports in the third column the approximate computing times (in seconds) for one ML fit with fixed tuning parameters as well as one round of hyperparameter tuning, and this for random forest, XGBoost, and LightGBM. As well known, both the XGBoost and LightGBM algorithms are fast with default tuning parameters, requiring only about one second to train. Random forest is, in comparison, relatively slow, with seven seconds to train. However, the prohibitive cost of tuning is appalling. All algorithms need large multiples of computing time to tune the hyperparameters. In particular, the XGBoost and LightGBM algorithms with many tuning hyperparameters are too long to optimize.[6]

To conclude, the random forest algorithm is preferred among the considered ML algorithms because the default tuning parameters yield good accuracy levels close to the tuned ones. This is a great advantage when faced with computational constraints such as given by the platform environments we consider in this paper.

5.2. Results for the bicycle sharing platform application

Section 5.2.1 discusses the overall performance of the forecast reconciliation methods on the Citi Bike dataset, and Section 5.2.2 details two sensitivity analyses on the choice of temporal aggregation orders in the reconciliation step.

[6] We use Bayesian optimization to tune the hyperparameters of XGBoost and LightGBM. Other methods or only subsets of parameters could be tuned. This would make the tuning faster.

Table A.1
London forecast reconciliation results for the MASE accuracy index.

Base Forecasts	Forecast Method	Delivery Area			Zone			London		
		30min	hour	day	30min	hour	day	30min	hour	day
Naive	Base	0.9824	0.9637	0.8748	0.9263	0.9027	0.8124	0.8456	0.8375	0.7903
	Random forest	0.7669	0.7821	0.8216	0.8025	0.8187	0.8087	0.8740	0.8667	0.8132
	XGBoost	0.8362	0.8427	0.8522	0.8538	0.8573	0.8326	0.8976	0.8826	0.8310
	LightGBM	0.8052	0.8147	0.8405	0.8338	0.8407	0.8267	0.8984	0.8827	0.8283
ETS	Base	0.9092	0.9974	0.7213	1.2484	1.3634	0.6884	1.7459	1.7079	0.6707
	Bottom Up	0.9092	1.0082	1.5758	1.1840	1.3201	1.7864	1.6391	1.6819	1.8977
	tcs	0.8847	0.9704	1.4441	1.1357	1.2613	1.6694	1.5493	1.5908	1.7722
	cst	0.8709	0.9518	1.3841	1.0890	1.2048	1.5389	1.4509	1.4903	1.6093
	ite	0.8871	0.9766	1.4746	1.1451	1.2772	1.7080	1.5677	1.6159	1.8118
	oct	0.8843	0.9675	1.4111	1.1337	1.2554	1.6346	1.5403	1.5770	1.7325
	Random forest	0.7849	0.8130	0.9005	0.8624	0.9061	0.9303	1.0235	1.0362	0.9552
	XGBoost	0.8827	0.9275	1.0175	0.9096	0.9464	0.9355	0.9807	0.9863	0.8968
	LightGBM	0.8291	0.8682	0.9886	0.8778	0.9175	0.9376	0.9868	0.9929	0.9117
SARIMA	Base	0.8875	0.9662	0.7708	1.1520	1.2731	0.7226	1.6034	1.6229	0.7441
	Bottom Up	0.8875	0.9744	1.4648	1.1624	1.2948	1.7398	1.6316	1.6738	1.8658
	tcs	0.8670	0.9430	1.3555	1.1181	1.2395	1.6223	1.5476	1.5882	1.7350
	cst	0.8536	0.9251	1.2891	1.0752	1.1882	1.5007	1.4538	1.4946	1.5827
	ite	0.8677	0.9471	1.3779	1.1239	1.2518	1.6539	1.5589	1.6063	1.7651
	oct	0.8641	0.9366	1.3144	1.1026	1.2171	1.5645	1.5163	1.5532	1.6797
	Random forest	0.7771	0.7986	0.8464	0.8258	0.8600	0.8384	0.9455	0.9590	0.8317
	XGBoost	0.8759	0.9039	0.9472	0.8739	0.8951	0.8371	0.9026	0.9060	0.7779
	LightGBM	0.8161	0.8440	0.9046	0.8331	0.8585	0.8130	0.8895	0.8956	0.7582
Forecast Combination	Base	0.8273	0.8622	0.7310	0.9582	1.0197	0.6855	1.2171	1.2186	0.6848
	Bottom Up	0.8273	0.8661	1.1351	0.9517	1.0196	1.2810	1.2049	1.2286	1.3533
	tcs	0.8199	0.8557	1.0893	0.9370	1.0019	1.2392	1.1738	1.1989	1.3105
	cst	0.8139	0.8468	1.0510	0.9151	0.9744	1.1657	1.1216	1.1464	1.2201
	ite	0.8220	0.8599	1.1101	0.9456	1.0158	1.2689	1.1939	1.2242	1.3408
	oct	0.8174	0.8505	1.0582	0.9262	0.9856	1.1948	1.1483	1.1688	1.2615
	Randomforest	0.7654	0.7816	0.8671	0.8129	0.8412	0.9145	0.9303	0.9380	0.9442
	XGBoost	0.8346	0.8455	0.8710	0.8435	0.8546	0.8533	0.8917	0.8912	0.8487
	LightGBM	0.8026	0.8146	0.8533	0.8209	0.8351	0.8435	0.8857	0.8856	0.8366

Notes: This table shows forecast accuracy measured in mean absolute scaled error(MASE). The forecast horizon is one week ahead. The best forecast reconciliation results for each base forecast (Naive, ETS, SARIMA, Forecast Combination) are highlighted in gray.

5.2.1. Overall forecast performance

Table 6 summarizes the WAPE forecast accuracy for the H3 cells on the complete temporal hierarchy for the different combinations of base forecasts and reconciliation methods; the results for the New York City market are displayed in Table 7.[7]

The ML-based forecast reconciliation methods generally result in substantial forecast accuracy improvements compared to the linear reconciliation methods, which have the largest improvement for ETS base forecasts. Furthermore, unlike the linear reconciliation methods, the ML-based forecast reconciliation methods are considerably more stable for different base forecasts. Similar to the other platform applications, ML-based reconciliation with random forest performs best overall and is, in fact, the only reconciliation method that offers a consistent improvement over the (already coherent) naive base forecasts. For example, the H3 cells random forest WAPEs with SARIMA base forecasts for the highest and lowest temporal frequencies are 0.2605 and 0.1589, compared to 0.3174 and 0.1760, respectively, for oct linear reconciliation. These results carry over for the market level results with random forest 0.2143 and 0.1490, and oct 0.2749 and 0.1651, respectively. Combining the base forecasts yields sizable gains only for the linear reconciliation methods. Changing the metric to the MASE forecast accuracy index instead of the WAPE, similar conclusions can be drawn; see Tables A.3 and A.4 in Appendix A for details.

5.2.2. Sensitivity analyses on temporal aggregation orders

The main results for Citi Bike are obtained when using the compact features matrix $\widehat{\boldsymbol{X}}_{b,\mathcal{V}}$ across all ML models, thereby using all $p = 10$ temporal frequencies but only for the focal bottom-level series for which the ML-model is trained. This entails two direct questions: (1) Is there a gain in forecast accuracy when using the complete features matrix $\widehat{\boldsymbol{X}}_{\mathcal{V}}$ instead of the compact one $\widehat{\boldsymbol{X}}_{b,\mathcal{V}}$? (2) Is there a gain in forecast accuracy for certain temporal frequencies of interest, for instance, 30-min, 1-h, and 24-h, when using all ten temporal frequencies for reconciliation instead of only those of three of interest? We next address these questions through sensitivity analyses.

[7] The ML-based forecast reconciliation results are reported for fixed choices of tuning parameters. For random forest and LightGBM, tuning does not result in improved forecast performance; for XGBoost, it leads to a minor improvement of 1.86% on average.

Table A.2
Manchester forecast reconciliation results for the MASE accuracy index.

Period	Forecast Method	Delivery Area			Market		
		30min	hour	day	30min	hour	day
Before shift	Base	0.8717	0.9537	0.8450	1.4371	1.5384	0.9517
	Bottom Up	0.8717	0.9618	1.4621	1.4855	1.6004	1.8757
	tcs	0.8490	0.9265	1.3518	1.4082	1.5088	1.7642
	cst	0.8417	0.9144	1.3061	1.3685	1.4599	1.6792
	ite	0.8503	0.9275	1.3595	1.4143	1.5143	1.7824
	oct	0.8512	0.9288	1.3497	1.4093	1.5117	1.7493
	Random forest	0.7792	0.8063	0.8768	0.9346	0.9677	0.8945
	XGBoost	0.8809	0.9206	1.0030	0.9271	0.9518	0.8713
	LightGBM	0.8259	0.8642	0.9915	0.9206	0.9501	0.8921
During shift	Base	0.9219	1.0624	0.8995	1.5797	1.7296	0.6676
	Bottom Up	0.9219	1.0740	1.8529	1.6131	1.7767	2.0856
	tcs	0.8940	1.0309	1.6943	1.4969	1.6401	1.8706
	cst	0.8859	1.0170	1.6178	1.4309	1.5610	1.7293
	ite	0.8988	1.0377	1.7151	1.5152	1.6587	1.9058
	oct	0.8877	1.0214	1.6469	1.4652	1.6090	1.8169
	Randomforest	0.7977	0.8658	1.1074	0.8420	0.8677	0.6978
	XGBoost	0.8939	0.9771	1.2160	0.8479	0.8551	0.6717
	LightGBM	0.8242	0.8986	1.1424	0.8319	0.8448	0.7108
After shift	Base	0.7874	0.8700	0.5977	1.3320	1.4557	0.5892
	Bottom Up	0.7874	0.8820	1.3808	1.3576	1.4816	1.7568
	tcs	0.7652	0.8486	1.2104	1.2565	1.3596	1.5810
	cst	0.7602	0.8387	1.1369	1.2075	1.3005	1.4573
	ite	0.7687	0.8536	1.2140	1.2724	1.3757	1.6030
	oct	0.7598	0.8413	1.1807	1.2280	1.3296	1.5263
	Randomforest	0.6525	0.6664	0.6148	0.7296	0.7405	0.5354
	XGBoost	0.7340	0.7598	0.7115	0.7427	0.7363	0.5554
	LightGBM	0.6854	0.7091	0.6905	0.7211	0.7208	0.5389

Notes: This table shows forecast accuracy measured in weighted absolute percentage error (MASE) for SARIMA base forecasts. The best forecast reconciliation results for the periods Before shift (before June 14, 2023), After shift (after July 14, 2023), and During shift (period in between) are highlighted in gray. Table 2 defines the linear benchmark methods.

Table B.1 in Appendix B addresses the first question and presents relative WAPEs when using the complete features matrix instead of the compact one. A value below one indicates better performance when using the complete features matrix. For the H3 cells, see the top panel of Table B.1; using the complete features matrix improves forecast accuracy for all base and ML forecast methods. In contrast, when turning to the results for the whole market, see bottom panel of Table B.1, the results remain either virtually unaffected or deteriorate. Hence, the relative performance of the ML-based reconciliation method with the complete versus compact features matrix is application-specific.

Table B.2 provides an answer to the second question. It reports relative WAPEs for the temporal frequencies of interest (30-min, 1-h, and 24-h) when using all ten temporal frequencies for reconciliation compared to using only those three. A value below one indicates better performance when using the full set rather than the reduced set of temporal granularities. We consistently find forecast performance to improve when using the full set for both the linear and the ML-based reconciliation methods. The improvement is typically larger for the ML-based reconciliation methods than the linear ones, at least for the bottom level in the cross-temporal hierarchy (30-min H3 cells). Note that the base forecasts and the bottom-up reconciliation method remain unaffected under this choice but are reported for completeness. This finding supports using intermediate temporal frequencies in the forecast reconciliation procedure beyond the frequencies of interest. However, it comes with the cost of additional computing time for the ML-based reconciliation methods.

6. Conclusion

Forecast reconciliation based on linear methods is useful and successful in various applications. Recently, ML-based forecast reconciliation methods have been proposed for cross-sectional time series hierarchies. We extend the latter approach to cross-temporal hierarchies. We describe the methodology and applications to unique platform datasets from an on-demand logistics company and a bicycle-sharing system. In particular, the datasets consist of demand streams that constitute a rich hierarchy in spatial and time dimensions, with bottom-level time series defined at 30-min frequency.

Our key empirical finding is that ML-based forecast reconciliation for our platform datasets can result in substantial forecast accuracy improvements compared to existing linear reconciliation methods. However, unless one can face the huge computational cost of hyperparameter tuning, ML-based reconciliation is not uniformly superior to linear methods. Another key finding is that, as data

Table A.3
Citi Bike forecast reconciliation results for the H3 cells for the MASE accuracy index.

Base Forecasts	Forecast Method	Temporal Frequency									
		30min	1h	1.5h	2h	3h	4h	6h	8h	12h	24h
Naive	Base	1.0521	1.0544	1.0573	1.0596	1.0612	1.0677	1.0693	1.0756	1.0838	1.0894
	Random forest	0.9883	1.0030	1.0099	1.0112	1.0135	1.0151	1.0032	0.9886	1.0017	0.9797
	XGBoost	1.0878	1.0996	1.1018	1.1025	1.0979	1.0969	1.0798	1.0559	1.0682	1.0334
	LightGBM	1.0653	1.0816	1.0862	1.0895	1.0873	1.0913	1.0694	1.0577	1.0654	1.0302
ETS	Base	1.4020	1.3554	1.3757	1.3809	1.3639	1.2197	1.1765	1.1796	1.1408	0.8585
	Bottom Up	1.4020	1.4561	1.4847	1.5013	1.5163	1.5216	1.5216	1.5272	1.5378	1.4401
	tcs	1.2976	1.3438	1.3680	1.3818	1.3887	1.3827	1.3802	1.3616	1.3551	1.2208
	cst	1.3254	1.3784	1.4051	1.4212	1.4358	1.4345	1.4373	1.4278	1.4327	1.3147
	ite	1.3337	1.3854	1.4114	1.4269	1.4402	1.4394	1.4410	1.4346	1.4399	1.3182
	oct	1.3118	1.3582	1.3826	1.3961	1.4033	1.3977	1.3950	1.3806	1.3764	1.2424
	Random forest	0.9687	0.9795	0.9838	0.9832	0.9793	0.9702	0.9458	0.9496	0.9462	0.9296
	XGBoost	1.0483	1.0571	1.0548	1.0538	1.0473	1.0408	1.0111	1.0111	1.0066	0.9757
	LightGBM	1.0193	1.0326	1.0339	1.0350	1.0296	1.0274	0.9964	0.9918	0.9960	0.9642
SARIMA	Base	1.2908	1.2452	1.2528	1.2680	1.1935	1.0547	0.9969	0.9857	0.8651	0.8587
	Bottom Up	1.2908	1.3389	1.3626	1.3761	1.3778	1.3732	1.3529	1.3471	1.3231	1.2031
	tcs	1.1860	1.2196	1.2351	1.2411	1.2279	1.2037	1.1789	1.1578	1.1169	0.9793
	cst	1.1874	1.2230	1.2390	1.2441	1.2316	1.2062	1.1790	1.1643	1.1213	0.9836
	ite	1.1863	1.2201	1.2349	1.2399	1.2256	1.1989	1.1706	1.1527	1.1101	0.9703
	oct	1.1859	1.2192	1.2351	1.2416	1.2284	1.2045	1.1796	1.1584	1.1183	0.9809
	Random forest	0.9676	0.9780	0.9801	0.9806	0.9730	0.9559	0.9256	0.9291	0.9194	0.8938
	XGBoost	1.0392	1.0488	1.0443	1.0434	1.0313	1.0228	0.9883	0.9888	0.9882	0.9483
	LightGBM	1.0082	1.0195	1.0200	1.0202	1.0103	1.0022	0.9677	0.9691	0.9643	0.9275
Forecast Combination	Base	1.0942	1.0615	1.0664	1.0744	1.0440	0.9599	0.9541	0.9418	0.8970	0.8459
	Bottom Up	1.0942	1.1268	1.1424	1.1536	1.1631	1.1600	1.1551	1.1558	1.1574	1.0833
	tcs	1.0409	1.0664	1.0785	1.0862	1.0895	1.0785	1.0722	1.0652	1.0542	0.9696
	cst	1.0589	1.0885	1.1024	1.1122	1.1183	1.1107	1.1049	1.1023	1.0946	1.0179
	ite	1.0695	1.0986	1.1125	1.1223	1.1281	1.1215	1.1153	1.1141	1.1080	1.0314
	oct	1.0385	1.0635	1.0755	1.0824	1.0850	1.0730	1.0665	1.0588	1.0468	0.9587
	Random forest	0.9316	0.9414	0.9452	0.9483	0.9489	0.9460	0.9285	0.9363	0.9360	0.9178
	XGBoost	1.0174	1.0261	1.0248	1.0251	1.0193	1.0207	0.9991	1.0049	1.0063	0.9821
	LightGBM	0.9961	1.0099	1.0105	1.0136	1.0115	1.0136	0.9904	1.0009	1.0045	0.9774

Notes: This table shows forecast accuracy measured in mean absolute scaled error (MASE). The best forecast reconciliation results for each base forecast (Naive, ETS, SARIMA, Forecast Combination) are highlighted in gray. Table 2 defines the linear benchmark methods.

shifts potentially impact platform data streams, our approach can react swiftly to such instability compared to linear approaches.

Several extensions should be considered. First, applications to other datasets that consist of different cross-temporal hierarchies. Second, our ML algorithms in the platform application consist of tree-based methods only, though our approach can be used with any ML algorithm. Considering computing time, testing alternative neural network-based methods using our approach would be interesting. Third, we currently retrain the ML algorithms at every iteration of the rolling window, and it would be interesting to investigate if online ML methods that incrementally update the forecast function as new data becomes available can lead to further computational efficiencies.

CRediT authorship contribution statement

Jeroen Rombouts: Conceptualization, Formal analysis, Methodology, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Marie Ternes:** Conceptualization, Data curation, Formal analysis, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Ines Wilms:** Conceptualization, Formal analysis, Funding acquisition, Methodology, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The Citi Bike data and the code to reproduce the results in this paper are available from https://github.com/MarieTernes/ml-crosstemporal-reconciliation. Due to a confidentiality agreement we are not allowed to share the data from the last-mile logistics platform application.

Acknowledgments

We thank the Editor, Associate Editor, and Referee for their constructive and detailed comments, which substantially improved the quality of the manuscript. We are also very grateful to Benjamin Wolter, Pablo Perez Piskunow,

Table A.4
Citi Bike forecast reconciliation results for the market for the MASE accuracy index.

Base Forecasts	Forecast Method	Temporal Frequency									
		30min	1h	1.5h	2h	3h	4h	6h	8h	12h	24h
Naive	Base	1.0962	1.0970	1.0989	1.1011	1.1026	1.1088	1.1124	1.1226	1.1300	1.1368
	Random forest	1.0760	1.0720	1.0706	1.0674	1.0645	1.0613	1.0489	1.0318	1.0370	1.0186
	XGBoost	1.1294	1.1222	1.1169	1.1119	1.1034	1.0988	1.0868	1.0664	1.0646	1.0399
	LightGBM	1.1380	1.1321	1.1290	1.1270	1.1194	1.1211	1.1018	1.0822	1.0809	1.0497
ETS	Base	1.8760	1.5007	1.4829	1.4706	1.4051	1.2403	1.1622	1.1718	1.1097	0.8897
	Bottom Up	1.6934	1.7012	1.7100	1.7126	1.7005	1.6991	1.6914	1.6866	1.7156	1.6726
	tcs	1.5184	1.5215	1.5262	1.5262	1.5066	1.4905	1.4813	1.4330	1.4347	1.3672
	cst	1.5325	1.5366	1.5406	1.5402	1.5226	1.5053	1.4980	1.4514	1.4620	1.3818
	ite	1.5684	1.5736	1.5784	1.5783	1.5620	1.5485	1.5402	1.5058	1.5238	1.4488
	oct	1.5475	1.5518	1.5571	1.5571	1.5393	1.5257	1.5166	1.4754	1.4891	1.4147
	Random forest	1.0518	1.0400	1.0360	1.0283	1.0126	0.9974	0.9691	0.9593	0.9691	0.9530
	XGBoost	1.0877	1.0765	1.0701	1.0626	1.0459	1.0359	1.0060	0.9939	1.0071	0.9792
	LightGBM	1.0722	1.0622	1.0561	1.0525	1.0341	1.0281	0.9942	0.9873	1.0004	0.9863
SARIMA	Base	1.4241	1.3784	1.3821	1.3950	1.2470	1.0891	1.0604	1.0529	0.9037	0.9511
	Bottom Up	1.3988	1.3967	1.3964	1.3937	1.3598	1.3250	1.3018	1.2380	1.2006	1.1089
	tcs	1.3385	1.3347	1.3317	1.3260	1.2867	1.2465	1.2145	1.1630	1.1091	1.0042
	cst	1.3236	1.3187	1.3152	1.3067	1.2641	1.2209	1.1852	1.1464	1.0817	0.9708
	ite	1.3393	1.3346	1.3314	1.3235	1.2818	1.2403	1.2039	1.1626	1.0997	0.9936
	oct	1.3396	1.3358	1.3330	1.3271	1.2885	1.2481	1.2176	1.1653	1.1123	1.0076
	Random forest	1.0460	1.0319	1.0252	1.0174	0.9957	0.9753	0.9419	0.9377	0.9292	0.9098
	XGBoost	1.0529	1.0383	1.0281	1.0208	1.0034	0.9937	0.9601	0.9518	0.9568	0.9235
	LightGBM	1.0414	1.0286	1.0190	1.0101	0.9918	0.9800	0.9496	0.9437	0.9444	0.9174
Forecast Combination	Base	1.2718	1.1534	1.1475	1.1505	1.0788	0.9933	0.9871	0.9771	0.9168	0.9045
	Bottom Up	1.2289	1.2324	1.2330	1.2357	1.2257	1.2148	1.2013	1.1833	1.1873	1.1493
	tcs	1.1703	1.1716	1.1719	1.1726	1.1604	1.1407	1.1308	1.1039	1.0927	1.0417
	cst	1.1876	1.1895	1.1897	1.1909	1.1801	1.1608	1.1502	1.1281	1.1186	1.0641
	ite	1.2159	1.2188	1.2199	1.2222	1.2121	1.1949	1.1834	1.1661	1.1617	1.1101
	oct	1.1654	1.1663	1.1669	1.1669	1.1542	1.1338	1.1250	1.0961	1.0825	1.0299
	Random forest	1.0026	0.9950	0.9920	0.9904	0.9829	0.9722	0.9531	0.9649	0.9559	0.9567
	XGBoost	1.0369	1.0276	1.0240	1.0207	1.0132	1.0012	0.9836	0.9913	0.9849	0.9813
	LightGBM	1.0307	1.0227	1.0177	1.0197	1.0084	1.0027	0.9794	0.9897	0.9904	0.9795

Notes: This table shows forecast accuracy measured in mean absolute scaled error (MASE). The best forecast reconciliation results for each base forecast (Naive, ETS, SARIMA, Forecast Combination) are highlighted in gray. Table 2 defines the linear benchmark methods.

Table B.1
Citi Bike forecast reconciliation results: sensitivity analysis features matrix.

Base Forecasts	Forecast Method	Temporal Frequency									
H3 cells results		30 min	1 h	1.5 h	2 h	3 h	4 h	6 h	8 h	12 h	24 h
Naive	Random forest	0.9975	0.9985	0.9979	0.9996	0.9988	1.0019	0.9945	0.9960	0.9952	0.9936
	XGBoost	0.9871	0.9895	0.9878	0.9866	0.9856	0.9887	0.9827	0.9914	0.9875	0.9846
	LightGBM	0.9883	0.9899	0.9897	0.9886	0.9894	0.9932	0.9888	0.9909	0.9942	0.9954
ETS	Random forest	0.9752	0.9729	0.9725	0.9726	0.9719	0.9713	0.9705	0.9690	0.9747	0.9782
	XGBoost	0.9595	0.9613	0.9620	0.9603	0.9605	0.9570	0.9520	0.9462	0.9569	0.9536
	LightGBM	0.9599	0.9584	0.9585	0.9562	0.9568	0.9522	0.9497	0.9565	0.9564	0.9616
SARIMA	Random forest	0.9861	0.9827	0.9842	0.9850	0.9848	0.9896	0.9870	0.9948	0.9983	1.0038
	XGBoost	0.9803	0.9829	0.9840	0.9819	0.9842	0.9789	0.9819	0.9816	0.9793	0.9903
	LightGBM	0.9841	0.9867	0.9869	0.9871	0.9896	0.9909	0.9868	0.9953	0.9952	1.0061
Forecast Combination	Randomforest	0.9882	0.9864	0.9852	0.9851	0.9837	0.9889	0.9865	0.9884	0.9888	0.9924
	XGBoost	0.9752	0.9754	0.9731	0.9736	0.9721	0.9705	0.9652	0.9705	0.9600	0.9640
	LightGBM	0.9700	0.9696	0.9684	0.9676	0.9680	0.9670	0.9640	0.9662	0.9644	0.9738
Market results		30 min	1 h	1.5 h	2 h	3 h	4 h	6 h	8 h	12 h	24 h
Naive	Randomforest	1.0191	1.0194	1.0203	1.0210	1.0173	1.0212	1.0118	1.0155	1.0144	1.0102
	XGBoost	1.0351	1.0351	1.0369	1.0367	1.0357	1.0396	1.0281	1.0372	1.0460	1.0511
	LightGBM	1.0245	1.0265	1.0267	1.0269	1.0265	1.0274	1.0184	1.0285	1.0357	1.0411

(*continued on next page*)

Table B.1 (*continued*).

Base Forecasts	Forecast Method	Temporal Frequency									
ETS	Random forest	1.0041	1.0021	1.0016	1.0015	1.0039	0.9997	1.0001	1.0055	0.9951	1.0033
	XGBoost	0.9889	0.9903	0.9913	0.9923	0.9948	0.9868	0.9860	0.9849	0.9775	0.9780
	LightGBM	0.9987	0.9985	0.9991	0.9960	1.0025	0.9888	0.9952	0.9974	0.9905	0.9812
SARIMA	Random forest	1.0262	1.0239	1.0248	1.0275	1.0291	1.0310	1.0311	1.0325	1.0336	1.0357
	XGBoost	1.0381	1.0431	1.0458	1.0461	1.0450	1.0343	1.0402	1.0443	1.0273	1.0387
	LightGBM	1.0468	1.0500	1.0523	1.0558	1.0562	1.0524	1.0509	1.0512	1.0461	1.0542
Forecast Combination	Random forest	1.0137	1.0131	1.0134	1.0149	1.0104	1.0180	1.0149	1.0115	1.0146	1.0082
	XGBoost	1.0172	1.0177	1.0147	1.0192	1.0106	1.0227	1.0109	1.0151	1.0171	0.9953
	LightGBM	1.0209	1.0211	1.0222	1.0208	1.0182	1.0202	1.0191	1.0152	1.0198	1.0175

Notes: This table shows relative forecast accuracy measured in weighted absolute percentage error (WAPE) when using the complete features matrix instead of the compact features matrix. A value below one indicates better performance when using the former.

Table B.2
Citi Bike forecast reconciliation results: sensitivity analysis temporal frequencies.

Period	Forecast Method	H3 cells			Market		
		30 min	1 h	24 h	30 min	1 h	24 h
Naive	Base	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	Random forest	0.9708	0.9750	0.9874	0.9674	0.9739	0.9924
	XGBoost	0.9426	0.9503	0.9217	0.9453	0.9551	0.9538
	LightGBM	0.9409	0.9459	0.9294	0.9529	0.9614	0.9615
ETS	Base	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	Bottom Up	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	tcs	0.9464	0.9419	0.8682	0.9256	0.9239	0.8648
	cst	0.9318	0.9270	0.8581	0.9164	0.9148	0.8487
	ite	0.9306	0.9254	0.8540	0.9235	0.9221	0.8671
	oct	0.9550	0.9515	0.8910	0.9330	0.9316	0.8777
	Random forest	0.9218	0.9163	0.9544	0.9003	0.8967	0.9739
	XGBoost	0.8984	0.8987	0.8865	0.8911	0.8947	0.9371
	LightGBM	0.8930	0.8893	0.8809	0.8840	0.8865	0.9468
SARIMA	Base	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	Bottom Up	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	tcs	0.9558	0.9516	0.9140	0.9749	0.9750	0.9528
	cst	0.9534	0.9497	0.9123	0.9685	0.9683	0.9447
	ite	0.9537	0.9492	0.9094	0.9759	0.9760	0.9579
	oct	0.9556	0.9516	0.9070	0.9751	0.9750	0.9516
	Random forest	0.8958	0.8881	0.9228	0.8553	0.8494	0.9141
	XGBoost	0.8720	0.8719	0.8829	0.8339	0.8351	0.9014
	LightGBM	0.8687	0.8627	0.8637	0.8271	0.8281	0.8941
Forecast Combination	Base	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	Bottom Up	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
	tcs	0.9588	0.9536	0.9047	0.9510	0.9492	0.9067
	cst	0.9470	0.9416	0.8932	0.9394	0.9376	0.8958
	ite	0.9476	0.9418	0.8944	0.9463	0.9449	0.9104
	oct	0.9700	0.9665	0.9292	0.9669	0.9654	0.9306
	Random forest	0.9588	0.9586	0.9720	0.9337	0.9345	0.9548
	XGBoost	0.9276	0.9296	0.9092	0.9038	0.9063	0.9117
	LightGBM	0.9302	0.9316	0.9117	0.9080	0.9118	0.9041

Notes: This table shows relative forecast accuracy measured in weighted absolute percentage error (WAPE) when using all ten temporal frequencies for reconciliation compared to using only the temporal frequencies of interest. A value below one indicates better performance when using the former.

Roger Caminal, and Afonso Rodrigues for expert advice, to Rob Hyndman, Artem Prokhorov, and Roberto Renò for comments provided on earlier versions of the paper, and to the participants of the IMS International Conference on Statistics and Data Science (ICSDS 2023) for helpful discussions. The last author was financially supported by the Dutch Research Council (NWO) under grant number VI.Vidi.211.032.

Appendix A. Platform applications: Forecast performance in terms of MASE

See Tables A.1–A.4.

Appendix B. Citi Bike sensitivity analyses

See Tables B.1 and B.2.

Appendix C. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ijforecast.2024.05.008.

References

Abolghasemi, M., Tarr, G., & Bergmeir, C. (2024). Machine learning applications in hierarchical time series forecasting: Investigating the impact of promotions. *International Journal of Forecasting*, *40*(2), 597–615.

Anderer, M., & Li, F. (2022). Hierarchical forecasting with a top-down alignment of independent-level forecasts. *International Journal of Forecasting*, *38*(4), 1405–1414.

Athanasopoulos, G., Ahmed, R. A., & Hyndman, R. J. (2009). Hierarchical forecasts for Australian domestic tourism. *International Journal of Forecasting*, *25*(1), 146–166.

Athanasopoulos, G., Hyndman, R. J., Kourentzes, N., & Panagiotelis, A. (2024a). Editorial: Innovations in hierarchical forecasting. *International Journal of Forecasting*, *40*(2), 427–429.

Athanasopoulos, G., Hyndman, R. J., Kourentzes, N., & Panagiotelis, A. (2024b). Forecast reconciliation: A review. *International Journal of Forecasting*, *40*(2), 430–456.

Athanasopoulos, G., Hyndman, R. J., Kourentzes, N., & Petropoulos, F. (2017). Forecasting with temporal hierarchies. *European Journal of Operational Research*, *262*(1), 60–74.

Caporin, M., Di Fonzo, T., & Girolimetto, D. (2023). Exploiting intra-day decompositions in realized volatility forecasting: A forecast reconciliation approach. arXiv preprint arXiv:2306.02952.

Di Fonzo, T., & Girolimetto, D. (2023a). Cross-temporal forecast reconciliation: Optimal combination method and heuristic alternatives. *International Journal of Forecasting*, *39*(1), 39–57.

Di Fonzo, T., & Girolimetto, D. (2023b). Spatio-temporal reconciliation of solar forecasts. *Solar Energy*, *251*, 13–29.

Di Fonzo, T., & Girolimetto, D. (2024). Forecast combination-based forecast reconciliation: Insights and extensions. *International Journal of Forecasting*, *40*(2), 490–514.

Girolimetto, D., Athanasopoulos, G., Di Fonzo, T., & Hyndman, R. J. (2023). Cross-temporal probabilistic forecast reconciliation: Methodological and practical issues. *International Journal of Forecasting*, in press.

Girolimetto, D., & Di Fonzo, T. (2023a). Foreco: Point forecast reconciliation. URL https://cran.r-project.org/package=FoReco, R package version 0.2.6.

Girolimetto, D., & Di Fonzo, T. (2023b). Point and probabilistic forecast reconciliation for general linearly constrained multiple time series. *Statistical Methods & Applications*, in press.

Hollyman, R., Petropoulos, F., & Tipping, M. E. (2021). Understanding forecast reconciliation. *European Journal of Operational Research*, *294*(1), 149–160.

Hoover, J. (2006). Measuring forecast accuracy: Omissions in today's forecasting engines and demand-planning software. *Foresight: The International Journal of Applied Forecasting*, *4*, 32–35.

Hyndman, R. J. (2006). Another look at forecast-accuracy metrics for intermittent demand. *Foresight: The International Journal of Applied Forecasting*, *4*(4), 43–46.

Hyndman, R. J., Ahmed, R. A., Athanasopoulos, G., & Shang, H. L. (2011). Optimal combination forecasts for hierarchical time series. *Computational Statistics & Data Analysis*, *55*(9), 2579–2589.

Hyndman, R. J., & Athanasopoulos, G. (2014). Optimally reconciling forecasts in a hierarchy. *Foresight: The International Journal of Applied Forecasting*, *35*, 42–48.

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: principles and practice*. 3rd. Melbourne, Australia: OTexts.

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, *22*(4), 679–688.

Hyndman, R. J., Lee, A. J., & Wang, E. (2016). Fast computation of reconciled forecasts for hierarchical and grouped time series. *Computational Statistics & Data Analysis*, *97*, 16–32.

Januschowski, T., Wang, Y., Torkkola, K., Erkkilä, T., Hasson, H., & Gasthaus, J. (2022). Forecasting with trees. *International Journal of Forecasting*, *38*(4), 1473–1481.

Komisarczyk, K., Kozminski, P., Maksymiuk, S., Kapsner, L. A., Spytek, M., Krzyzinski, M., et al. (2024). treeshap: Compute SHAP values for your tree-based models using the 'TreeSHAP' algorithm. URL https://CRAN.R-project.org/package=treeshap, R package version 0.3.1.

Koning, A. J., Franses, P. H., Hibon, M., & Stekler, H. (2005). The M3 competition: Statistical tests of the results. *International Journal of Forecasting*, *21*(3), 397–409.

Kourentzes, N., & Athanasopoulos, G. (2019). Cross-temporal coherent forecasts for Australian tourism. *Annals of Tourism Research*, *75*, 393–409.

Kourentzes, N., Svetunkov, I., & Schaer, O. (2023). Tsutils: Time series exploration, modelling and forecasting. URL https://CRAN.R-project.org/package=tsutils, R package version 0.9.4.

Lundberg, S. M., Erion, G. G., & Lee, S. I. (2018). Consistent individualized feature attribution for tree ensembles. arXiv preprint arXiv:1802.03888.

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems: vol. 30*, (pp. 4765–4774). Curran Associates, Inc..

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, *36*(1), 54–74.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022a). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, *38*(4), 1346–1364.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022b). The M5 competition: Background, organization, and implementation. *International Journal of Forecasting*, *38*(4), 1325–1336.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022c). Predicting/hypothesizing the findings of the M5 competition. *International Journal of Forecasting*, *38*(4), 1337–1345.

Medeiros, M. C., Vasconcelos, G. F. R., Veiga, Á., & Zilberman, E. (2021). Forecasting inflation in a data-rich environment: The benefits of machine learning methods. *Journal of Business & Economic Statistics*, *39*(1), 98–119.

Molnar, C. (2022). *Interpretable Machine Learning* (2nd ed.).

Panagiotelis, A., Athanasopoulos, G., Gamakumara, P., & Hyndman, R. J. (2021). Forecast reconciliation: A geometric view with new insights on bias correction. *International Journal of Forecasting*, *37*(1), 343–359.

R Core Team (2023). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing, URL https://www.R-project.org/.

Shapley, L. S., et al. (1953). A value for n-person games. In *Contributions to the theory of games: vol. 2*, (pp. 307–317). Princeton University Press Princeton.

Spiliotis, E., Abolghasemi, M., Hyndman, R. J., Petropoulos, F., & Assimakopoulos, V. (2021). Hierarchical forecast reconciliation with machine learning. *Applied Soft Computing*, *112*, Article 107756.

Theodosiou, F., & Kourentzes, N. (2021). Forecasting with deep temporal hierarchies. Available at SSRN 3918315.

Van Erven, T., & Cugliari, J. (2015). Game-theoretically optimal reconciliation of contemporaneous hierarchical time series forecasts. In A. Antoniadis, J.-M. Poggi, & X. Brossat (Eds.), *Modeling and stochastic learning for forecasting in high dimensions* (pp. 297–317). Springer.

Wang, S., Zhou, F., Sun, Y., Ma, L., Zhang, J., & Zheng, Y. (2022). End-to-end modeling of hierarchical time series using autoregressive transformer and conditional normalizing flow-based reconciliation. In *2022 IEEE international conference on data mining workshops* (pp. 1087–1094). IEEE.

Wickramasuriya, S. L., Athanasopoulos, G., & Hyndman, R. J. (2019). Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization. *Journal of the American Statistical Association*, *114*(526), 804–819.